
Technical and Scientific Audit

Detecting failure in chronic intracortical brain–computer interface decoders

A complete, evidence-anchored review of the repository, its data, its code, its statistics, its retracted claims and its readiness for competition — together with a findings-driven design for the original data-collection component.

Repository	gnair40/bci-ews-research
Branch audited	claude/isef-research-pipeline-9zt4uq
HEAD commit at audit	07cb392767a101321e30c99f679ecb94e0ed70cf
Commits in history	123 (first 34dcb05, 16 Aug 2026; last 07cb392, 5 Sep 2026)
Working tree	clean at time of audit
Audit date	6 September 2026
Audit method	Direct inspection of the working tree, the full git history, and re-execution of the project's own verification gates
Verification re-run	93 / 93 headline claims reproduced from data files

STANDARD OF EVIDENCE

This report was produced by inspecting primary evidence in the repository: files, git objects, commit messages, JSON manifests, checksums, and the output of re-running the project's scripts. Where a fact could not be recovered from that evidence, it is labelled **UNRECOVERABLE** rather than inferred. Nothing in this report is invented. Section 0 states the audit's own limits before any finding is presented.

Contents follow. Sections 1–15 correspond to the audit scope requested; Section 0 records the audit's method and limits; Appendix A lists every verified claim.

Contents

0. How this audit was carried out, and what it cannot see	5
What was actually executed during this audit	5
Four things this audit cannot establish	5
0.3 Register of unrecoverable information	6
1. Repository inspection	7
1.1 Physical inventory at HEAD	7
1.2 Branches, tags, and what that implies	7
1.3 Churn: which files were rewritten most	8
1.4 Structural finding: raw and processed data are genuinely separated	8
2. Complete project timeline	9
2.1 Commits per day, across the whole project	9
2.2 The five phases, and what closed each one	9
2.3 Frozen artefacts, with their timestamps	10
3. Claude Code activity audit	11
3.1 What is recoverable, and what is not	11
3.2 The commit gaps, and how each is accounted for	11
3.3 Errors the assistant made, and how each was caught	12
3.4 What the error pattern shows	13
4. What the research actually is	14
4.1 The problem the project exists to solve	14
4.2 The hinge: why observational data cannot answer the original question	14
4.3 Scientific question, engineering goal, and hypotheses — kept separate	14
4.4 Variables	15
4.5 The five gates, and why silence comes first	15
5. Dataset audit	17
5.1 The archived dataset — provenance	17
5.2 What is actually in it	17
5.3 Verification performed on the archived data	18
5.4 The constructed dataset — the fault corpus	18
5.5 The corpus's own limitations, as measured	19
6. Codebase audit	20
6.1 The pipeline, in execution order	20
6.2 Dependencies and environment	20

6.3 The detector contract — the design decision that prevents leakage	21
6.4 decoder-guard — how the monitor is built	21
6.5 Code-quality findings	21
7. Analysis and statistics audit	22
7.1 Statistical methods used, and where	22
7.2 The unit-of-analysis error, in full	22
7.3 The three error classes, now encoded as a linter	23
7.4 Multiplicity, and the discipline that governs it	23
8. Failed approaches and abandoned directions	25
8.1 Directions abandoned, with the evidence that closed each	25
8.2 The five withdrawn claims	25
8.3 Predictions recorded in advance, and how they turned out	26
8.4 One approach that failed and is preserved as a bound	26
9. Results, reconstructed from the data	28
9.1 Phase 1–2: the preregistered negative result	28
9.2 Phase 3: the benchmark	28
9.3 But the information is present	29
9.4 The monitor beats the trivial comparator, and exactly where it was designed to	29
9.5 The clean positive: task changes are not mistaken for faults	29
9.6 Why nothing can pass: the arithmetic, then the cause	30
9.7 The day effect: the finding nobody was looking for	31
9.8 Calibration efficiency — the first clean, non-null positive	31
9.9 Attribution: a capability that was built, scored, and nearly thrown away	32
9.10 What the project ruled out, and what remains	32
10. Reproducibility audit	33
10.1 What the gate checks, and its state on re-run	33
10.2 What the gate caught on its first run	33
10.3 The audit's own three bugs	33
10.4 The two files a static audit cannot resolve	34
10.5 Reproducibility in practice: can a stranger run this?	34
10.6 What reproducibility here does not mean	34
11. Scientific rigour audit	35
11.1 Rigour scorecard	35
11.2 Contradictions and tensions found	35
11.3 Methodological problems that remain open	36
11.4 What a hostile reviewer would attack, and whether it holds	36

12. ISEF readiness	38
12.1 Rules and safety	38
12.2 Research plan components	38
12.3 One document the project depends on and does not contain	39
12.4 How this presents to judges	39
12.5 The presentation risk nobody has addressed yet	40
13. The original data-collection component, derived from the findings	41
13.1 The chain, stated in the required form	41
13.2 The mandatory findings → experiment table	41
13.3 The apparatus	42
13.4 Variables for the rig study	43
13.5 The rule that governs what may be simulated	44
13.6 Designs considered and rejected	44
13.7 Ranking, risks, and what must be decided first	44
13.8 What the integrated project becomes	45
14. Where the project stands, and what to do next	46
14.1 Current state, in one table	46
14.2 Prioritised roadmap	46
14.3 The three most important next steps	47
15. Project memory / master context	48
15.1 What the project is, in five sentences	48
15.2 Standing constraints set by the researcher	48
15.3 The facts a future session must not have to rediscover	48
15.4 Working practices this project follows	49
15.5 Claims: what may and may not be quoted	49
15.6 If the project has to be defended in one paragraph	50
Appendix A — every verified claim, recomputed	51
Appendix B — sources the project cites	54

SECTION 0 · METHOD AND LIMITS

0. How this audit was carried out, and what it cannot see

Every statement in the sections that follow rests on one of four kinds of evidence, and the kind is named wherever the distinction matters.

Evidence class	What it is	How it was obtained	Trust
Working-tree artefact	A file present at HEAD — a script, a report, a JSON manifest, a checksum.	cat, ls, direct read	High — it is the thing itself
Git-history artefact	A commit message, a diff, a deletion, a rename, an author date.	git log, git show, git log --diff-filter	High, but records the author's account of intent
Recomputed result	A number regenerated by running the project's own code now.	python3 scripts/31_verify_claims.py and the four other gates	High — reproduced during this audit
Audit judgement	An assessment of rigour, risk or readiness that no file states.	Reasoning over the three classes above	Marked as judgement in the text

What was actually executed during this audit

The audit did not take the repository's reports at their word. Five automated gates were re-run against the working tree, and all five pass:

Gate	Script	Result on re-run
Claim verifier — recomputes every headline number from the data files	31_verify_claims.py	93 / 93 match — “All 93 headline claims match the data files.”
Reproducibility audit — imports, producers, ordering, references	55_reproducibility_audit.py	PASS — 6 imports declared, 35 consumed files all produced, no ordering inversions
Claims register — status of every claim, cross-checked against the verifier	56_claims_register.py	38 entries , 92 of 93 verifier checks cited by an entry
Statistical hygiene linter — three known error classes over all scripts	61_statistical_hygiene.py	PASS — 0 unreviewed sites, 8 deliberate sites documented
Log coverage — every commit date accounted for in the research log	65_log_coverage.py	PASS — 12 commit dates, 10 logged directly, 2 reconciled with a reason

Four things this audit cannot establish

Stated first, so that nothing later is read as stronger than it is.

- **It cannot verify the raw data against Dryad today.** The download manifest records a SHA-256 of 6d12b5db... for a 411,951,588-byte archive, and that digest matches what Dryad reported at download time on 25 August 2026. This audit confirms the manifest exists and is internally consistent; it did not re-download 411 MB to re-verify the hash against the live repository.
- **It cannot detect a script that runs and produces a wrong answer.** The reproducibility gate is static analysis; the claim verifier checks that reported numbers match stored files, not that the computation behind those files is correct. The project's own report says this in the same words, and it is the honest limit of self-checking.
- **It cannot recover reasoning that was never written down.** Where a decision appears in a commit or a log entry, this report quotes it. Where it does not, the gap is marked **UNRECOVERABLE** in Section 3 rather than reconstructed.
- **It cannot assess the project against the 2026–27 ISEF forms,** which had not been issued at the time the plan was drafted. Section 12 assesses readiness against the structural requirements that are stable across form years and

flags precisely which parts are form-year-specific.

0.3 Register of unrecoverable information

Consolidated here so it is not scattered. Each item is something a reader might reasonably expect the repository to answer and which it does not.

#	What cannot be recovered	Why	Does it affect a result?
U-1	The “literature review” the project repeatedly cites	Referenced and quoted in at least six files, and stated in the ISEF plan to be in the repository. No commit ever added it.	No result, but the framing. Its conclusions survive in the files that quote it; its reasoning does not. See 12.5.
U-2	Which party proposed each scientific option	The repository records what was decided and why, not who proposed it. Some log entries do name a researcher instruction; the attribution is not systematic.	No. It affects the AI-disclosure narrative, not any number.
U-3	The number of working sessions	12 commit dates exist and 54 dated log headings, several sharing a date with markers like “(later)” and “(night)”. Session boundaries were never timestamped.	No.
U-4	What happened between 16 and 25 August 2026	Eight days with no commits and no log entry spanning them.	No.
U-5	The exact conversation prompts	Transcripts are not part of the repository artefact. Only outcomes are.	No.
U-6	The T11 arm of the combination study, as originally run	Lost to a container restart. Recorded in commit a713c83 rather than hidden.	No — it was re-run.
U-7	Byte-level re-verification of the raw archive against Dryad today	This audit did not re-download 411 MB. The SHA-256 in the manifest matches what Dryad reported at download time.	No, unless Dryad's deposit has changed since v6.

HEADLINE AUDIT JUDGEMENT

One general finding shapes the whole audit and is stated here so that it is not mistaken for flattery later: **this project’s most valuable property is that it repeatedly caught itself being wrong, and left the wreckage in place.** Five claims are formally withdrawn. Three published p-values were corrected from significant to null. Four separate checking tools were themselves buggy before they were useful, and each of those bugs is documented in the tool's own report. That behaviour is rarer and harder than a positive result.

SECTION 1

1. Repository inspection

The repository is a single-purpose research repository. There is no application code, no service, no deployment; every file exists to acquire data, analyse it, check the analysis, or record what happened.

1.1 Physical inventory at HEAD

Category	Count	Location	What it holds
Analysis scripts	65	scripts/01-65_*.py	17,807 lines of Python, numbered in the order they were written
Research documents	28	research/	Preregistrations, frozen designs, the project definition, the ISEF plan, the log
Reports	45 Markdown + 3 HTML	reports/	One report per study, plus two rendered demonstrations and a plain-language guide
Figures	17 PNG	reports/figures/	Session timeline, performance over time, error distribution, missingness, decoder stability, EWS controls and power sweep, the MINDFUL reproduction, the two “why negative” panels, AUC by fault mode, calibration and staleness curves, decision rules. Tracked in git only since commit e421166 ; previously gitignored, so every image link in every report was broken on GitHub
Rendered write-ups	3 HTML	reports/	project_guide.html (plain-language guide), monitor_demo.html (live demonstration, including the two cases it gets wrong), and bad_day_problem.html, which is also published as a shareable artefact
Processed data	136 files	data/processed/	CSV and JSON intermediates; all tracked so the verifier runs on a fresh clone
Raw data	166 files, 825 MB extracted	data/raw/	The Dryad archive, its extraction, and the download manifest — never modified

1.2 Branches, tags, and what that implies

Property	State	Audit note
Branches	claude/isef-research-pipeline-9zt4uq (working), main	All 123 commits are on the working branch. No feature branches, no merges.
Tags	None	Finding. Nothing is tagged — not the design freeze, not the Phase 1–2 closeout, not the Phase 3 completion. The freeze is recoverable only because FROZEN_DESIGN.json stores the commit hash inside itself. A tag would make the same fact visible from git tag. See Section 14, roadmap item R7.
Deleted files	1	reports/BENCHMARK_T5_T5_local_trailing.md removed in commit 4dee00d. A duplicated-prefix filename from a parameterisation bug, not a discarded result.
Renames	3	All documentation reorganisation; no analysis output was renamed.
Working tree at audit	Clean	No uncommitted changes, no untracked analysis output.
Missing artefact	1	Finding. A “literature review” is cited as an existing document in at least six files and is stated in the ISEF plan to be “in the project repository”. It is not, and no commit ever added it. See Section 12.3.

1.3 Churn: which files were rewritten most

Churn is a fingerprint of where the thinking happened. The three most-touched files are exactly the three that ought to be, if the project is what it claims to be.

File	Commits touching it	Reading
research/research_log.md	58	Nearly one in two commits updates the log. The log is not a retrofit.
README.md	25	Rewritten repeatedly as the question changed — including commit 6c27b21, “Rewrite the README, which still described the abandoned question.”
scripts/31_verify_claims.py	19	The verifier grew from a handful of checks to 93, one study at a time. Each addition is a commit that also added the study it checks.

1.4 Structural finding: raw and processed data are genuinely separated

The user’s standing constraint was that raw downloaded data must stay separate from processed data and must never be modified. This audit confirms the separation is real and enforced by structure, not merely by intention:

- data/raw/ contains the Dryad ZIP, its extracted tree, the upstream README, and download_manifest.json. Nothing in the pipeline writes there after 01_download_dataset.py.
- data/processed/ contains every derived artefact. The reproducibility gate independently confirms that all 35 consumed processed files have a producing script.
- The fault injector never writes a modified copy of the raw arrays. It writes an **injection plan** — episode identifiers, onsets, seeds — and applies the transformation in memory at scoring time. This is why 1,850 fault episodes cost kilobytes rather than gigabytes, and why the raw files are provably untouched.

SECTION 1 FINDING

The one structural weakness found in Section 1 is the absence of tags. It costs nothing scientifically today, but it is the difference between “the design was frozen at 20db485, and here is the file that says so” and “git show design-freeze”. For a judge, the second is one command.

SECTION 2

2. Complete project timeline

Reconstructed from commit author-dates, the dated headings in `research/research_log.md`, and the timestamps embedded in the frozen design and injection plans. Every date below is from a git object or a file, not from memory.

2.1 Commits per day, across the whole project

Date	Commits	What happened that day
16 Aug 2026	6	Repository created. README, research plan, learning roadmap, initial log. No data yet.
— 17–24 Aug —	0	Eight days with no commits. No commits. Cause is UNRECOVERABLE from the repository; no log entry spans it.
25 Aug 2026	18	The single heaviest day. Network policy fixed, Dryad download script written against the real API, dataset downloaded and SHA-256 verified, loader built, exploration run, fixed-decoder claim tested twice, EWS controls and power sweep, MINDFUL baseline reproduced at $r = 0.985$, deterioration definitions compared, freeze mechanism built.
26 Aug 2026	11	The hinge day. Design frozen (20db485, 01:31 UTC), observable selected, two amendments recorded, preregistered analysis run — negative, drift characterised, Phases 1–2 closed, project redefined backward from its final product, Amendment 1 (no human participants) recorded, fault injector built.
27 Aug 2026	1	Reference decoder frozen; corpus severities recalibrated against it. Four bugs found that the corpus would otherwise have hidden.
28 Aug 2026	24	The largest day. Grading system built before the monitor; decoder-guard written; first benchmark (every detector fails the silence gate); local re-baselining; causal detrending; T5 transfer; full 36- then 48-configuration benchmark; task-change test; achievability; threshold-bound correction; the v2 prediction recorded and falsified; decision-rule comparison; the aggregation limit.
30 Aug 2026	1	One-measurement-per-session finding lands. Logged under the 28 August heading — the session crossed midnight.
31 Aug 2026	4	Phase 3 consolidated into one report; the “participant” terminology note; feature study preregistered; live demonstration built.
1 Sep 2026	9	README rewritten; documents verified against data; feature study frozen, implemented, run — null; an overclaim fixed that rendering the demo exposed.
2 Sep 2026	11	Combination study (null); calibration efficiency (the first non-null positive); staleness; day variance with the fault-mix bug fixed; the first preregistered positive, P5.
3 Sep 2026	11	Abstention (null); day effect across detectors; the ceiling challenge that forced a retraction; unambiguous episodes; per-day chance; severity ladder; window overlap; reproducibility audit; claims register; attribution accuracy.
4 Sep 2026	15	Mode separability; per-mode detection; the pooled-window sweep that unpublished three results; the hygiene linter; figures and processed files un-gitignored.
5 Sep 2026	12	Invariant detector (worse); invariant attribution (worse); original data-collection design recorded; log coverage gate; published write-up brought current.

2.2 The five phases, and what closed each one

Phase	Dates	Question it asked	How it closed
0 — Scoping	16 Aug	What is this project going to be?	A plan and a roadmap. Superseded on 26 Aug.
1 — Foundation	25 Aug	Is the data real, is it usable, does my pipeline work?	Dataset verified by checksum; published result reproduced at $r = 0.985$; detector validated on synthetic systems with known answers.

Phase	Dates	Question it asked	How it closed
2 — The preregistered test	25–26 Aug	Do early-warning signals precede decoder decline?	Negative, and overturned by its own prespecified limitation check. The indicator rose faster during health ($\tau = +0.857$) than overall ($\tau = +0.743$).
3 — Build the ground truth	26 Aug – 1 Sep	If observational data cannot answer this, what can?	1,850 fault episodes with onsets locked under checksum; a grading system built before the monitor; 48 configurations benchmarked; 0 pass all five gates.
4 — Interrogate the negative	2–5 Sep	Why did it fail, and is the failure real or an artefact of my own method?	Twelve further studies, each preregistered. Five claims withdrawn, three p-values corrected, four checking tools debugged. The cause is located at lag-1 $r = 0.995$.

2.3 Frozen artefacts, with their timestamps

These are the objects that make the preregistration claim checkable. Each carries its own creation time and the commit it was created at.

Artefact	Frozen at (UTC)	At commit	What it locks
research/FROZEN_DESIGN.json	2026-08-26 01:31:10	20db485	Phase 1–2 analysis: participant, variable, level, method, boundary day 758, $p = 0.0018$, plus SHA-256 of <code>blocks.csv</code> and <code>trials.csv</code> . Two amendments, both dated before the analysis.
data/processed/injection_plan.json	2026-08-27 17:48:12	6edf505	T11: 1,073 episodes, master seed 20260826, onsets in the 0.25–0.55 window of each block, per-episode seeds. <code>episodes_sha256 = 37faf7dc....</code> One amendment, with its reason recorded.
data/processed/injection_plan_T5.json	2026-08-28 02:14:02	f2d6efc	T5: 777 episodes, same seed and design. <code>episodes_sha256 = 3c96508c....</code> No amendments.
research/FEATURE_STUDY_FROZEN.json	1 Sep 2026	0950c04	Four feature families named and the list closed; success criterion fixed at +0.05 AUC on both participants in the same direction.
research/COMBINATION_STUDY_FROZEN.json	2 Sep 2026	32aa623	How the four-component decomposition generalises to new feature families.
research/DETECTOR_V2_RECORD.json	28 Aug 2026	4cf5f57	A prediction about detector v2, committed while the results files were untouched. Outcome: satisfied on T11, falsified on T5, not adopted .

WHY THE TIMELINE MATTERS

The ordering in 2.3 is the load-bearing fact of the whole project. The injection plan — which fixes **when every fault begins** — was written on 27–28 August. The grading harness came after it (commit e161594, “Build the grading system before the monitor exists”), and decoder-guard itself came after that (bf5e8c0, “Add decoder-guard, written before the baseline scores were read”). The ground truth could not have been tuned to the detector, because the detector did not exist. That ordering is visible in `git log`, not merely asserted.

SECTION 3

3. Claude Code activity audit

This section audits what the AI assistant did in this repository, what it got wrong, and what cannot be recovered. It is included because ISEF treats undisclosed or misdescribed AI use as grounds for disqualification, and because an honest account of tool use is itself part of the paper trail.

3.1 What is recoverable, and what is not

Question	Answer	Source
Who authored the commits?	All 123 commits are authored Claude . The commits were made by the assistant under the researcher's direction, in the researcher's repository.	<code>git log --format=%an</code>
Were the scientific decisions the assistant's or the researcher's?	Mixed, and the split is partly UNRECOVERABLE. The repository records <i>what</i> was decided and <i>why</i> , in commit messages and the log. It does not record which party proposed each option. Several log entries do name a researcher instruction — for example Amendment 1 opens “The researcher has established that human participants are not available” — but that attribution is not systematic.	Log headings; PROJECT_DEFINITION.md Amendment 1
How many working sessions were there?	UNRECOVERABLE as a count. Twelve distinct commit dates exist. The log has 54 dated headings, some of which are marked “(later)”, “(evening)”, “(late)”, “(night)” on the same date — so sessions are more numerous than dates, but the boundaries are not logged as timestamps.	<code>grep "^## " research/research_log.md</code>
Did any work happen that left no trace?	At least once, yes, and it is recorded. Commit a713c83 notes the T11 arm of the combination study “lost to a container restart”. The ephemeral execution environment can lose in-flight work; the repository is the only durable record.	Commit a713c83
What reasoning shaped the original framing?	Partly UNRECOVERABLE. The project repeatedly cites a “literature review” as the source of its framing, its threat model and its comparator choice, and quotes it directly. That document is not in the repository and was never committed. Its <i>conclusions</i> survive in the files that quote it; its reasoning does not.	<code>grep -rn "literature review"; git log --all --diff-filter=A</code>
What were the exact prompts?	UNRECOVERABLE. Conversation transcripts are not in the repository and are not part of the artefact. Only their outcomes are.	—

3.2 The commit gaps, and how each is accounted for

Gaps in a research record invite the question “what happened there?”, so they are enumerated rather than smoothed over.

Gap	Length	Status
17–24 Aug 2026	8 days, no commits	UNRECOVERABLE. The log jumps from “Day 1” on 16 Aug straight to dataset acquisition on 25 Aug. No entry explains the interval.
29 Aug 2026	1 day, no commits	Bracketed by heavy days on 28 and 30 Aug. Not separately explained; consistent with a rest day.
30 Aug 2026	Commits, no matching log heading	Reconciled, not hidden. The 30 Aug commits belong to the session logged as “28 August 2026 — One measurement per session”. The session crossed midnight.
31 Aug 2026	Commits, no matching log heading	Reconciled. Split across three headings: two dated 28 August and one dated 1 September.

A CORRECTION THAT WAS DELIBERATELY NOT MADE

The handling of the last two entries is worth naming as a rigour finding. The obvious fix — re-dating the log headings to match the commits — was **explicitly refused**. Instead, `scripts/65_log_coverage.py` was written to compare commit dates against log headings and to carry a RECONCILED table naming each mismatch and its reason. Re-dating would have made the record *look* tidier while destroying the evidence that sessions ran past midnight. The gate now fails if any future commit date is neither logged nor reconciled.

3.3 Errors the assistant made, and how each was caught

Fourteen distinct errors are recoverable from commits and reports. They are listed in full because the pattern — and specifically **how** each was caught — is more informative than the count.

#	Error	Consequence had it stood	How it was caught
1	Fault-mix control compared a float to a string (e.g. <code>severity</code> vs <code>"high"</code>), so the test was always False.	A control that measured nothing looked exactly like a control that passed. Fixing it changed a conclusion : T5 is confounded, $p = 0.886$.	Noticed because every day reported precisely 0.00
2	Inverse-variance weighting applied to a bounded statistic (AUC).	A pooled AUC inflated from 0.675 to 0.836 by over-weighting extreme days; $p = -0.913$ between $ AUC-0.5 $ and its standard error.	Noticed because the number was implausibly good
3	<code>prob_better_than_chance</code> stored an inverted quantity: <code>mannwhitneyu</code> returns U_1 , so $U_1/(nm)$ is $P(\text{error exceeds chance})$.	A decoder 55% better than chance was recorded as 0.273 and would have been read as its probability of beating chance.	Found while checking an already-published recommendation
4	Every p-value in <code>ACHIEVABILITY.md</code> pooled windows within sessions.	Sample size inflated 26.6× ; four statistics published as exactly $p = 0$; three results significant as published are not significant , including one at $p = 3.4e-15$ that is actually 0.144.	Noticed while decomposing a headline
5	The reproducibility audit had three bugs of its own: f-strings parsed by regex (<code>filename }</code> .csv), filenames held in variables, and the audit listed as its own producer.	Its first run reported 27 problems, of which 1 was real. A tool that reports its own artefacts converts “unchecked” into “checked and clean”.	Manual inspection of its own findings
6	The statistical hygiene linter missed its own motivating example, because <code>26_achievability.py</code> wraps <code>mannwhitneyu</code> in a local helper.	The check would have certified the exact file it was written to catch.	Asking “does it find the thing I already know is there?”
7	A headline figure — the 11-of-13 sign test, $p = 0.011$ — was computed in an ad-hoc shell one-liner while the verifier checked its JSON.	A fresh clone would have failed that claim. A verified claim resting on an unreproducible file looks exactly like a verified claim.	The reproducibility audit, first run
8	An edit to <code>57_attribution_accuracy.py</code> consumed its JSON write.	Two claims would have had no backing file.	The claim verifier, with two KeyErrors
9	Dead line in <code>64_invariant_attribution.py</code> (<code>conf[(m, MAPPING and m)]</code> , immediately overwritten).	None — numbers confirmed unchanged after removal. Recorded anyway.	Code review before commit
10	Overclaim: “decoder error is measurable online, without labels.”	False — it needs the intended direction, which a deployed monitor does not have. The correction is appended inline to the report, not edited away.	Rendering the demo exposed it

#	Error	Consequence had it stood	How it was caught
11	The WARN threshold search stopped at 59 while scores reach 3,200.	Fifteen of 36 configurations were reported as “no operating point exists” when none existed <i>below an arbitrary bound</i> — a property of the search presented as a property of the detectors. Corrected to 1 of 48.	Checking the search range against observed scores
12	Wrong decoder API assumed (<code>load_decoder</code> , <code>block_error</code> do not exist).	A script that could not run.	Immediate execution failure
13	Gates run <i>after</i> committing left <code>data/processed/*.json</code> dirty.	The repository would carry outputs that disagreed with the committed code.	A stop hook. Order corrected to run gates, then commit.
14	Figures and processed outputs were gitignored: 17 figures and 126 data files.	Broken image links on GitHub, and a claim verifier that could not run on a fresh clone .	Checked by actually cloning the repository

3.4 What the error pattern shows

- **Four of fourteen errors were in the checking tools themselves** (#5, #6, #7, #8). The project names this explicitly — “the fourth time in this project that a checking tool was itself wrong before it was useful.” The lesson encoded in response is a good one: every new gate was tested against a defect already known to exist.
- **Three errors changed a published conclusion** (#1, #4, #11). None was quietly patched. Each correction is a separate commit whose message names the invalidated claim, and each affected report carries the correction **inline** rather than having the old text replaced.
- **The most dangerous class is the one that looks like success** — #1 and #5 both produced output that resembled a passing check. Both were caught by a human-style question about plausibility (“why is this exactly 0.00?”, “why is this implausibly good?”) rather than by any automated gate.
- **One class of error is not covered by any gate**: a computation that runs, produces a number, and is simply the wrong computation. The hygiene linter finds shapes, not wrong answers, and its own report says so.

AI USE AND THE DISCLOSURE REQUIREMENT

Audit judgement, stated as judgement: the AI-assistance question a judge will ask is not “did you use a tool?” but “do you understand what it produced well enough to have caught it being wrong?” This repository’s answer is fourteen documented instances of exactly that. The evidence is strong. What is **missing** is a single, plainly-worded AI-use disclosure written for the form — the ISEF plan has a §C-AI placeholder flagged for re-checking against the 2026–27 rules, but no final text. Section 12 carries this as a readiness gap.

SECTION 4

4. What the research actually is

This section states the project in its own terms — question, engineering goal, hypotheses, variables — drawn from research/PROJECT_DEFINITION.md and the frozen designs, not paraphrased from the reports.

4.1 The problem the project exists to solve

People with paralysis use intracortical brain–computer interfaces to control a cursor by imagining movement. Electrode arrays record neural activity; a **decoder** converts that activity into cursor motion. Over months and years the recording drifts — electrodes degrade, tissue responds, the neural code itself shifts — and the decoder silently becomes wrong. The current remedy is recalibration: stop, run a supervised block, refit. It costs the user time and it is scheduled by guesswork rather than by evidence of need.

The obvious research move is to look for an early-warning signal that precedes decline. **That is what Phases 1–2 did, and it is where the project found its real problem.**

4.2 The hinge: why observational data cannot answer the original question

THE ARGUMENT THAT REORGANISED THE PROJECT

Lead time = (time performance crossed threshold) – (time the monitor warned). In observational data the first term is **defined by the analyst**, so lead time is partly an artefact of that definition. And the false-alarm rate cannot be estimated at all, because there is no known-healthy period of controlled length with no impending event. **You cannot validate an early-warning system without a system in which you control, and therefore know, when the warning should have fired.**

Two things follow, and both are unusual for a high-school project. First, the negative result of Phases 1–2 is not a setback to be worked around — it is a statement about what any observational study of this kind can establish. Second, building a corpus with **known onset** stops being an optional “original data” add-on and becomes **logically necessary**: it is the only configuration in which the primary claim is checkable at all.

This argument is recorded in research/PROJECT_DEFINITION.md §1, written on 26 August 2026 — after Phases 1–2 and before any ISEF section was drafted. The document states its own purpose as working *backward* from the final product “so that every later choice is derived rather than assumed.”

4.3 Scientific question, engineering goal, and hypotheses — kept separate

These are three different things, and the project keeps them apart deliberately, so that an engineering goal cannot quietly become a hypothesis.

	Statement
Scientific question	When a neural control interface degrades, does the <i>statistical structure</i> of the input signal change measurably <i>before</i> closed-loop task performance changes — and if so, is that change distinguishable from the change caused by the user simply doing something different?
Engineering goal	Build a monitor that converts that statistical change into a warning state early enough to act on, at a false-alarm rate low enough to be worth having, and with an attribution specific enough to choose the right action.
H1 — lead time	For injected degradations of known onset, decoder-guard reaches WARN before performance crosses its degradation threshold, with median lead time > 0, at ≤ 1 false alarm per 10 hours of matched healthy operation.
H2 — specificity	Attribution accuracy across the four injected failure modes exceeds chance, and genuine task changes are classified TASK-CHANGE rather than as failures.

Statement	
H3 — transfer	The identical monitor, with no refitting of its structure, passes the five gates on the public dataset — in particular, it is <i>silent</i> during T11's healthy baseline, which the Phase 1–2 indicator was not.
H4 — impact	A recalibration policy driven by WARN performs no worse than fixed-schedule recalibration while performing significantly fewer recalibrations.
Falsification, fixed in advance	H1 is false if median lead time ≤ 0 , or if the false-alarm constraint cannot be met at any operating point. H3 is false if the monitor trends during healthy baseline. Any of these being false is a reportable result.

4.4 Variables

Class	Variable	Levels or definition
Independent	Failure mode	4 injected: RATE_LOSS, CHANNEL_DROPOUT, GAIN_DRIFT, GEOMETRY_ROTATION — plus NONE and TASK-CHANGE as controls
	Degradation rate	fast (0.10 of block), medium (0.35), slow (0.80)
	Severity at endpoint	benign / sub-threshold / threshold-crossing, per mode
	Onset time	Randomised in the 0.25–0.55 fraction of each block, drawn and logged before any detector existed
Dependent	Detection AUC	Window-level and episode-level; both reported, and the distinction is enforced by the claims register
	Median lead time	Seconds between WARN and the performance crossing. Never reported without its false-alarm rate
	False-alarm rate	Per hour of matched healthy operation
	Silence-gate pass	Fraction of healthy episodes on which the risk signal trends
	Attribution accuracy	Fraction of episodes whose dominant component names the true mode, against a measured chance level
Controlled	Decoder	Ridge regression to intended direction, frozen after fitting; adaptive normalisation deliberately disabled to isolate degradation from the compensator
	Windowing	30 s windows (1,500 bins at 20 ms), stepped 5 s (250 bins), identical across every configuration
	Detector information	A detector sees only the feature stream — never decoder output, task, performance, or fault label — and is fitted on healthy windows alone
	Splits	By block. One free parameter chosen on validation. Test set read once
Constant	Source data	MINDFUL deposit, Dryad version 6, never modified
	Random seed	Master seed 20260826; a derived per-episode seed stored with each episode
	Chance level	Measured by shuffled pairing per participant and per day, not assumed to be 90°

4.5 The five gates, and why silence comes first

The validation standard is the project's methodological product. It exists because Phase 1–2's failure was not a low p-value — it was a significant result that meant nothing.

#	Gate	Requirement	Why it exists
1	Silence	No significant trend during healthy operation	Checked first. Phase 1–2's indicator rose faster during health ($\tau = +0.857$) than overall ($+0.743$). A detector that is never quiet cannot warn.
2	Rate invariance	Not explained by mean firing rate	The Phase 1–2 indicator was 71% mean firing rate ($R^2 = 0.707$).
3	Comparator	Beat counting activity, not only the published baseline	Mean firing rate alone predicted performance as well as the whole pipeline ($\rho = -0.880$ against $+0.858$).

#	Gate	Requirement	Why it exists
4	Elapsed time	Survive controlling for time	Controlling for elapsed time, essentially nothing of the Phase 1–2 result remained.
5	Detrend	De-trend the series before testing it for a trend	The frozen design omitted this. It is why recording drift produced $p = 0.0002$. It was not corrected retrospectively — doing so after seeing the outcome would have destroyed the preregistration.

SECTION 5

5. Dataset audit

Two datasets exist in this project: one downloaded, one constructed. They are audited separately because their evidentiary status is completely different.

5.1 The archived dataset — provenance

Field	Value
Name	MINDFUL — “Measuring instability in chronic human intracortical neural recordings towards stable, long-term brain-computer interfaces”
Publication	Pun et al. (2024), <i>Communications Biology</i>
DOI	10.5061/dryad.n2z34tn5s — Dryad, version 6
Licence	CC0-1.0 — public domain dedication, no restriction on reuse
Downloaded	2026-08-25 18:14:05 UTC, by scripts/01_download_dataset.py against https://datadryad.org/api/v2
Files	2 — MINDFUL_Data.zip (411,951,588 bytes) and README.md (3,348 bytes)
Integrity	SHA-256 verified against the digest Dryad reported: 6d12b5dbc9cac654ff1d0679e9753bf042b56cd5b8852eb31236b3cdecf7332
Extracted	166 files, 825,449,877 bytes — 162 .mat, 3 .md, 1 .zip
Ethics	Collected years earlier by the depositing researchers under their own approvals. This project recruits nobody.

“PARTICIPANT” DOES NOT MEAN WHAT IT USUALLY MEANS

A terminology hazard that the repository addresses head-on, in commit 7102ff6 (“Say plainly that ‘participant’ never means a recruited person”): in this project **“participant” means whose published recording a data file contains** — T5, T11, T15 — using the depositing authors’ own convention. It does not mean a person this project recruits. “Adding a third participant” means downloading one more public file. Nobody is recruited, nobody wears anything, no consent form exists, and ISEF Form 4 is not expected to apply.

5.2 What is actually in it

	T11	T5	Note
Sessions (trial days)	15	6	Plus 2 extra T11 sessions on different tasks
Blocks used in the corpus	29	21	
Trials	1,839	1,200	3,301 in total including the extra sessions
Neural features per bin	384	192	T5 has no spike-power channels
Trial-day span	658–800 (142 days)	2121–2149 (28 days)	T11 is the longitudinal one
Bins at 20 ms	530,734	251,974	782,708 total — 4.35 hours of recording in all. Note: the bin counts cover every recorded block, including the two extra T11 sessions, while the trial counts above are the main cohort only — 3,301 trials in total

Recorded correction to the literature review: it had the two participants reversed. T11, not T5, is the longitudinal record. Found by inspecting the data rather than by re-reading the paper.

5.3 Verification performed on the archived data

Claim checked	Method	Outcome
The decoder in the published study is fixed	The paper's Methods, and independently from the data itself	Confirmed twice — and the Methods reveal more: weights are frozen but an adaptive normalisation layer runs continuously (3-minute rolling z-score for T11; bias correction at rate 0.3 for T5)
Performance genuinely degrades	Two independent measures, change-point located by three agreeing methods	T11 steps at trial day 758 ($p = 0.0018$); T5 dips and recovers
Task type is not a confound	Task name tabulated by session	One task per participant throughout the main cohort
The analysis pipeline is correct	Reproduce the published Figure 1b from the raw deposit	$r = 0.985$ against the published 0.985
The EWS detector works at all	Synthetic positive and negative controls with known answers	Fires on a bifurcation, stays silent on monotonic drift
The detector has adequate power	Power scaling against record length	0.25 $\rightarrow$ 1.00 as records lengthen

5.4 The constructed dataset — the fault corpus

This is the project's original scientific artefact, and it is the answer to §4.2. It does not exist anywhere publicly.

Property	T11 corpus	T5 corpus
Episodes	1,073	777
Blocks drawn from	29	21
Design	4 modes $\times$ 3 rates $\times$ 3 severities, balanced, plus NONE controls	Identical design
Onset window	0.25–0.55 of each block, uniformly drawn	Identical
Ramp	Linear; fast = 0.10 of block, medium = 0.35, slow = 0.80	Identical
Master seed	20260826, with a derived per-episode seed stored per episode	Same master seed
Plan created	2026-08-27 17:48:12 UTC at commit 6edf505	2026-08-28 02:14:02 UTC at commit f2d6efc
Checksum of the episode list	37faf7dc71ad43...	3c96508c1805ec...
Amendments	1, with its reason recorded in the file	0
Episodes whose injected fault actually did measurable damage	764 of 1,073	369 of 777
Split — by block , never by episode	test 851 / fit 148 / validation 74; 586 of the 851 test episodes crossed	test 444; 219 of them crossed

A distinction that matters and is easy to misread: “crossing” is one of the three **designed** severity levels (348 episodes on T11, 252 on T5), while “crossed” above means the fault, once applied, **actually pushed the frozen decoder past its degradation threshold** — an outcome, not a setting. On near-chance sessions a crossing-severity fault often does not cross, which is limitation L03.

The four injected failure modes, and what each is designed to test

Mode	What it does to the feature stream	Severities (benign / sub / crossing)	What it tests
RATE_LOSS	Scales all channels down together	0.10 / 0.25 / 0.55	The easy case — total activity falls, so counting spikes should win
CHANNEL_DROPOUT	Zeroes a nested set of channels	0.05 / 0.30 / 0.60	Nested across severities so the ladder is monotone — it was not, originally
GAIN_DRIFT	Per-channel gain change, mean-conserved per bin	0.20 / 0.50 / 1.20	Channels drift apart with total activity preserved — counting spikes must fail
GEOMETRY_ROTATION	Norm-preserving Givens rotations of the channel vector	0.15 / 0.45 / 1.20	Shape changes, magnitude does not — the strongest test of the comparator gate

Four bugs the corpus would have hidden, found before it was used

Bug	Why it mattered	Fix
GAIN_DRIFT was not mean-preserving — twice	Log-centring preserves the <i>geometric</i> mean, leaving +116% at crossing severity; pre-onset rescaling still left +20.8%. A mode designed to be invisible to a spike counter was loudly visible to one.	Conserved per bin: now −2.4%
CHANNEL_DROPOUT's severity ladder ran backwards	Each level drew an independent channel set, so a “more severe” level could remove easier channels.	Nested channel sets; ladder now monotone
The pre-onset guarantee held only by floating-point luck	A leak of post-onset data into the pre-onset window would let a detector “warn” off the leak itself — a perfect self-fulfilling result.	Enforced by construction
T5 ran with an empty test set	The harness reported nothing and did not complain.	The harness now refuses to run rather than reporting nothing

5.5 The corpus's own limitations, as measured

- **Severity is only valid in aggregate (L01).** The three levels are correctly ordered in 57% of session×mode cells on T11 and 67% on T5, and validity degrades as the session's baseline error rises ($p = -0.677$ and -0.926). This is the only relationship in the project that replicates significantly with the same sign on both participants.
- **Window overlap contaminates the healthy reference (L02).** Windows are 30 s and step 5 s, so six windows per episode that *start* before onset still *span* it. Measured cost: every AUC understates by 0.007 (T11) and 0.012 (T5). The bias is **conservative** — it pulls the reference toward the fault — and 11 of 13 days improve when it is removed (sign test, $p = 0.011$). No conclusion changes. The fix for a future run is stated: `require start + window <= onset_bin`.
- **On bad sessions the ground truth is nearly degenerate (L03).** Where the decoder is already near chance, an injected fault does almost no measurable damage. **6 of 13 T11 sessions cannot be scored on unambiguous faults at all**; day 783 retains 1 measurable fault of 20, day 672 retains none.
- **GEOMETRY_ROTATION at crossing severity clips 17% of entries** and loses 18% of mean activity, so at that level it stops being a clean test of the comparator gate.
- **Evaluation is open-loop.** An injected episode is a recording; it cannot adapt. Performance is therefore the frozen decoder's angular error against the participant's real recorded intended direction, not closed-loop task success. This is a direct consequence of Amendment 1 and is stated as a limitation, not discovered later.

WHAT THE TWO DATA SOURCES GIVE BETWEEN THEM

The honest pairing, which the project states itself: **recorded sessions have live human compensation but unknown onset; injected episodes have known onset but no live compensation.** Neither alone is sufficient. Every quantitative claim in the project must say which of the two it rests on — and the reports do.

SECTION 6

6. Codebase audit

65 scripts, 17,807 lines. They are numbered in the order they were written, which is mostly — but, as the project's own audit discovered, not entirely — the order they run in.

6.1 The pipeline, in execution order

Stage	Scripts	What it does
Acquire	01–04	Download from Dryad with SHA-256 verification; inspect the archive; load the nested .mat tree into tidy tables; produce the exploration report
Validate the method	05–10	Check the decoder is fixed; build synthetic EWS positive/negative controls; power sweep; record-length check; reproduce the published MINDFUL result ($r = 0.985$); design power analysis
Preregister	11–13	Compare six deterioration definitions using performance data only; freeze the design with data hashes and a git commit; select the observable
Phase 1–2 analysis	14–16	Run the preregistered analysis; diagnose the drift; synthesise
Build ground truth	17–18	The fault injector (onsets locked under checksum); the frozen reference decoder
Build the grader	19–21	The detector contract; the evaluation harness; the score report. Committed before the monitor existed — at that point the only detectors it could score were three baselines this project did not invent
Build the monitor	22–25	decoder - guard; benchmark figures; the 48-configuration matrix; the task-change test
Interrogate	26–30	Achievability; decision rules; the operating-point bound; the aggregation limit; the demo export
Preregistered follow-ups	32–54	Feature families; combination; transfer; calibration curve; staleness; day variance; pooling loss; day predictors; mediation; label-free signals; cross-detector day effect; ceiling challenge; unambiguous episodes; per-day chance; severity ladder; window overlap
Self-audit	55–65	Reproducibility audit; claims register; attribution accuracy; mode separability; per-mode detection; unit of analysis; hygiene linter; permutation-invariant features; invariant detector; invariant attribution; log coverage
Runs last	31 and 55	Out of numeric order, and now documented as such. 31_verify_claims.py reads output from fifteen higher-numbered scripts because it recomputes every headline from whatever the pipeline produced. Its number records when it was written, not where it runs.

6.2 Dependencies and environment

Package	Pin	Why it is here
requests	≥ 2.31	Downloading from the Dryad API over HTTPS
numpy	≥ 1.26	Numerical arrays — the foundation
pandas	≥ 2.0	Named-column tables
scipy	≥ 1.11	Scientific computing; .mat reading; the statistical tests
h5py	≥ 3.10	MATLAB v7.3 files, which are HDF5 underneath
matplotlib	≥ 3.8	Figures

Python 3.11.15. The reproducibility gate confirms all six third-party imports are declared. **Audit finding:** the pins are lower bounds ($\geq$), not exact versions, and the project's own audit report says so — “it does not check that requirements.txt pins versions, only that the packages are named.” A pip freeze lockfile would close this. See roadmap item R6.

6.3 The detector contract — the design decision that prevents leakage

SCRIPTS/19_DETECTORS.PY

A detector sees **only the feature stream**: channels × binned activity. It never sees the decoder's output, the task, the performance number, or any fault label. It is fitted on **healthy windows alone** — one-class. Three baselines are held to exactly the same standard.

This is not a stylistic choice. A supervised failure-predictor trained on labelled degradation invites exactly the leakage that would make a positive result worthless: the model learns the label, not the phenomenon. One-class fitting on healthy data is what makes the AUCs in Section 9 mean something. It is also what makes them low.

6.4 decoder-guard — how the monitor is built

Four named components, each calibrated against healthy data, with the largest reported as the risk score **and its cause named**. The system works in log space, so a uniform gain change becomes an additive shift and can be separated from a change in the shape across channels.

Component	What it measures	Intended fault signature
level	Overall activity relative to the healthy reference	RATE_LOSS
silence	Channels that have gone quiet	CHANNEL_DROPOUT
dispersion	Channels spreading apart with the total conserved	GAIN_DRIFT
profile	Residual shape across channels after the above	GEOMETRY_ROTATION — this mapping was withdrawn, see W04

Interpretability is a stated requirement rather than a bonus: every risk score decomposes into named, unit-carrying contributions, so “why is it warning?” has a sentence-shaped answer. The output is a risk score (0–1), a warning state (NOMINAL / WATCH / WARN / FAIL-LIKELY) with hysteresis and dwell so it cannot chatter, an attribution, and a confidence — where low confidence is **reported, not hidden**.

6.5 Code-quality findings

Finding	Severity	Detail
Numbering does not equal execution order	Documented, resolved	Discovered by the project's own audit. Renumbering would break the script names quoted throughout 53 documents, so the README now states the truth: two scripts run last.
Two files the static audit cannot resolve	Documented, hand-verified	reference_decoder.npz and .json are written through a decoder_paths() helper that <i>returns</i> the path, so the filename literal never appears near the write. Both are named in a two-item RESOLVED_BY_HAND allowlist. Anything not on that list is a real finding.
Eight deliberate statistical-pattern sites	Documented, reviewed	Six pooled-inference sites and two inverse-variance sites are kept on purpose — the pooled ones so the register can record what claim C02's window-level number is, and the inverse-variance ones because Cochran's Q requires them and they are already labelled . . . _BIASED_DO_NOT_QUOTE. Each is listed in a REVIEWED dictionary with its reason.
No unit tests	Real gap	Audit judgement. There is no test suite. Correctness is enforced by five output-level gates, by synthetic controls with known answers, and by the reproduce-a-published-result check — which is a stronger scientific defence than unit tests would be, but does not catch a silently wrong intermediate function. Errors #1 and #3 in Section 3.3 are exactly what a unit test would have caught.
No CI	Minor	The gates are run by hand before each commit. A stop hook caught one instance of them being run in the wrong order (error #13).

SECTION 7

7. Analysis and statistics audit

This is the section where a research project usually breaks. It is examined against the specific failure modes that make published results wrong: unit-of-analysis errors, multiplicity, post-hoc selection, biased estimators, and inference on non-independent observations.

7.1 Statistical methods used, and where

Method	Where	Audit note
Kendall's τ	The preregistered Phase 1–2 trend test	Rank-based — appropriate for a short, non-normal series
Permutation test (5,000)	The change-point locating deterioration	The boundary at day 758, $p = 0.0018$, agreed by three methods and two variables
Permutation on day labels	The staleness test	The important one. A naive test said $p = 0.003$; permuting day labels gives 0.128 . The naive test was measuring day-to-day variation, not staleness.
Mann–Whitney U / AUC	Every detection headline	Point estimates survive pooling; p-values do not — see 7.2
Bootstrap over episodes	All corrected confidence intervals	Episodes, not windows. 2,000 resamples, so the floor is $p = 0.0010$ and it is reported as a floor , not as zero
Spearman ρ and partial ρ	The day-predictor studies	Bonferroni-corrected at 0.01 for five preregistered predictors
Cochran's Q and I^2	Day-to-day heterogeneity	$I^2 = 0.856$ — only about a quarter of the day-to-day AUC spread is sampling noise
Sign test	Combining the window-overlap result across arrays	11 of 13 days improve; $p = 0.011$ one-sided
Lag-1 autocorrelation and effective n	The aggregation limit	The single most consequential number in the project

7.2 The unit-of-analysis error, in full

This is the project's largest statistical failure and its most instructive correction. It is presented in detail because a judge who understands it will trust everything else more.

WHAT WENT WRONG

26_achievability.py pooled every 5-second window from every episode and ran a Mann–Whitney test on the result. But claim C04 establishes that windows within a session are **not independent** — lag-1 $r = 0.995$. Pooling them inflates the apparent sample size by a measured **26.6×** (22,590 windows against 850 episodes). An AUC point estimate survives that; a p-value does not.

Participant / baseline / detector	AUC (window)	AUC (episode)	p published	p corrected
T11 · calibrate once · decoder_guard	0.491	0.504	0.0464	0.8480 (!)
T11 · calibrate once · distribution_shift	0.541	0.558	6.85e-20	0.0030
T11 · calibrate once · mean_activity	0.514	0.527	0.00138	0.2170 (!)
T11 · calibrate once · robust_dispersion	0.465	0.470	3.38e-15	0.1440 (!)
T11 · recent normal · decoder_guard	0.693	0.672	0	0.0010
T11 · recent normal · distribution_shift	0.666	0.646	1.17e-303	0.0010
T11 · recent normal · mean_activity	0.611	0.557	2.03e-137	0.0040

Participant / baseline / detector	AUC (window)	AUC (episode)	p published	p corrected
T11 · recent normal · robust_dispersion	0.602	0.578	8.05e-117	0.0010
T5 · recent normal · decoder_guard	0.708	0.742	0	0.0010
T5 · recent normal · distribution_shift	0.661	0.661	7.23e-205	0.0010
T5 · recent normal · mean_activity	0.612	0.589	8.5e-100	0.0010
T5 · recent normal · robust_dispersion	0.620	0.642	3.12e-114	0.0010

(!) = significant as published, not significant at the episode level. Regenerated by `scripts/60_unit_of_analysis.py`. Two rows (`decoder_guard_joint`) omitted here for width; the full table is in `reports/UNIT_OF_ANALYSIS.md`.

- **AUC point estimates barely moved:** median shift 0.0202, largest 0.0545. Pooling affects variance, not location.
- **Four statistics were published as exactly “p = 0”** and ten below 1e-20. All are ordinary numbers at the episode level.
- **Three results significant as published are not significant now.** The starkest: `robust_dispersion` at the calibrate-once baseline was published at $p = 3.4e-15$ and is actually $p = 0.144$. Fifteen orders of magnitude, from an effect that is not there.
- **No conclusion of the project rests on these p-values.** “0 of 48 configurations pass the gates” is a count; the operating-point bound is arithmetic; the aggregation limit is itself the reason the correction was needed. What changed is whether the supporting statistics are stated honestly.
- **The correction is inline.** `ACHIEVABILITY.md` carries a (!) correction block rather than having its numbers silently replaced.

7.3 The three error classes, now encoded as a linter

Error class	How it was found	What it cost	Now checked by
Inference on pooled, non-independent units	Noticed while decomposing a headline	4 p-values published as 0; 26.6× inflation; 3 false positives	<code>pooled_inference</code> — 0 unreviewed sites, 6 deliberate
A comparison that can never be true	Noticed because every day reported exactly 0.00	A control that measured nothing looked like a control that passed	<code>impossible_comparison</code> — 0 sites
An estimator biased by its own weights	Noticed because the number was implausibly good	A pooled AUC inflated 0.675 → 0.836	<code>inverse_variance</code> — 0 unreviewed, 2 deliberate

The linter failed its own first test. Its first version flagged one site and did **not** flag `26_achievability.py` — the very file it was written for — because that file wraps `mannwhitneyu` in a local `auc(pos, neg)` helper, so the pooled variable never appears in the test’s arguments. Fixing it (by detecting local functions that contain a test call) took the count from 1 site to 9, including all three known-bad lines. **A linter that cannot catch the case it was written for is worse than no linter, because it converts “unchecked” into “checked and clean.”**

Stated limits of the linter, from its own report: it is static, so it finds shapes rather than wrong answers; it knows three patterns, which are the three this project made; and `impossible_comparison` is a heuristic that would miss a numeric field never used arithmetically in the same file. “A fourth error class will not be caught until it is found some other way — which is the honest limit of learning from your own mistakes.”

7.4 Multiplicity, and the discipline that governs it

Risk	How the project handles it	Audit verdict
Trying many deterioration definitions and picking the best	Six definitions compared using performance data only , before any indicator existed; one frozen in advance in <code>deterioration_definition.md</code> .	Handled — and this was an explicit standing instruction from the researcher

Risk	How the project handles it	Audit verdict
Trying many features until one works	Four families named in advance, list closed , criterion frozen at commit 0950c04 with a checksum: +0.05 AUC on both participants in the same direction .	Handled, and it bit — F3 spectral cleared the bar on T11 (0.750) and failed badly on T5 (0.556). With one participant, or with the rule written afterwards, it would have been “the feature that worked.”
Five day-predictors tested at once	Bonferroni threshold set at 0.01; direction of each predictor committed in advance	Handled . P5 survives at $p = 0.0055$. P4 came out with the wrong sign (+0.582 against a predicted negative) and is reported as such, claiming nothing
48 benchmark configurations	No selection at all — every configuration is reported, and the headline is a count over all of them	Handled
Post-hoc structure in a null result	The F1/F2 finding in the feature study is labelled post-hoc in the report itself and in the register (status EXPLORATORY)	Handled by labelling

THE SINGLE STRONGEST PIECE OF STATISTICAL DISCIPLINE HERE

The both-participants rule deserves particular notice. It was written **before** the feature study and it is the reason the study returned a null. A weaker project would have reported F3 as a success on T11 and mentioned T5 in a limitations paragraph. This one recorded the criterion first and then obeyed it.

SECTION 8

8. Failed approaches and abandoned directions

These are preserved because they are evidence. A project that shows only what worked is indistinguishable from a project that tried many things and reported one.

8.1 Directions abandoned, with the evidence that closed each

Direction	Why it was tried	What closed it
Critical slowing down as the mechanism	The literature's standard early-warning theory: rising variance and rising autocorrelation before a bifurcation	Closed empirically, not by preference. No observable in the data has measurable memory (0.3–0.7 samples across 20 ms–5 s), so the autocorrelation half cannot be tested at all . And the premise is doubtful: the data show a monotonic 56.5% firing-rate decline into a saturating compensator, which is a ramp, not a fold bifurcation.
Predicting time-to-recalibration	Drops the CSD commitment, which is correct	Generic and underpowered: 2 participants, ~40 sessions, high leakage exposure. And it inherits the same circularity — still no ground truth for onset.
Combining many indicators	Multimodality usually helps	Mean firing rate alone already matched the whole five-dimensional pipeline. Combining confounded measures does not decontaminate them.
Better decision rules	After finding the information was present (AUC 0.69–0.71), the natural conclusion was that the failure was in the machinery converting scores to warnings	Refuted by measurement. Three rules compared by their full curve , not one operating point: threshold, CUSUM, and CUSUM with a specificity gate. Detection inside the budget: 0.0% for all three on T11.
Longer or more windows	The obvious way to buy precision	There is no $\sqrt{N}$ to collect. Effective independent samples per session: 0.1 (T11), 0.4 (T5).
Better session-level aggregators	If per-window AUC is 0.70 and a session holds 55 windows, aggregation should help	Six tried — median, mean, 90th percentile, max, top-decile mean, fraction above own median. None beats a plain median , and the tail-leaning ones are near chance (0.44–0.47).
Calibrate-once deployment	The deployment model everyone would want	Carries literally no information: AUC 0.491 when a monitor calibrated once is applied months later. That is a bound, not a shortfall of effort.
Abstention (“the monitor says when not to trust it”)	A natural response to the finding that accuracy varies enormously by session	Four label-free candidates, two stages. Nothing passes stage 2. Abstention is an untested proposal, not a capability.
EMG, EEG, consumer neuro-headsets, camera-tracked hand movement	Original data collection with a human in the loop	Ruled out by Amendment 1 (26 Aug 2026): no human participants, and data collected from oneself still counts as human-participant research.

8.2 The five withdrawn claims

Each of these was asserted, then retracted on evidence. The register marks them **WITHDRAWN — do not quote**, and they remain in the repository.

ID	What was claimed	What refuted it
W0 1	The monitor fails because the decoder has lost its signal	Margin over a session's own chance level does not predict monitor accuracy (+0.264, $p = 0.38$). Days 800 and 783 beat their own chance by near-identical margins with accuracies of 0.974 and 0.319 .
W0 2	A specific session property causes the accuracy swing	Trace noise predicts identically (−0.720) and correlates with decoder error at 0.813; the partials are identical in both directions (−0.333). Identical partials both ways is collinearity, not mediation.

ID	What was claimed	What refuted it
W03	Weaker faults on bad sessions explain the swing	Achieved damage barely relates to monitor accuracy (+0.181, $p = 0.55$). This confound was predicted to hold and did not.
W04	The preregistered mapping GEOMETRY_ROTATION → profile	Rotation is injected with norm-preserving Givens rotations, and dispersion is defined as channels spreading apart with the total conserved — the same signature by definition . The mapping was wrong from the start; re-ordering the rule cannot fix it.
W05	Dropout, gain drift and rate loss face an information ceiling	That was the linear model's limit, not the features'. With permutation-invariant summaries the confusable trio rises from 0.650 to 0.987 on T11 and 0.623 to 0.998 on T5. A 12-random-channel control reaches only 0.702, so the gain is representation, not dimension count.

8.3 Predictions recorded in advance, and how they turned out

Six predictions were committed to the repository before the analysis that would test them. This is the sharpest available evidence that results were not reverse-engineered.

Prediction	Recorded at	Outcome
Detector v2 will improve on the benchmark	4cf5f57, while the results files were untouched	Split — satisfied on T11, falsified on T5. The change was not adopted.
The day effect will be shared by all four detectors	63d6d9a	Wrong. It belongs to distribution-based detectors and is absent from scalar ones — a better answer than either option offered.
The achievable-damage ceiling confounds P5	f86f73e	Wrong (W03). The confound does not hold; P5 strengthens when controlled for damage (−0.794).
Removing GEOMETRY_ROTATION will drop the headline to 0.60–0.65	660620a	Wrong on both counts. It moves by 0.037 (T11) and 0.002 (T5).
A one-class detector on invariant features will detect better	f8e27d2	Wrong. Worse on both axes: AUC 0.617/0.680 against 0.672/0.742, and 98.5% of healthy episodes trending against a 31% best.
Label-free invariant attribution will beat the current rule	3f658dd	Wrong. 51.1%/39.7% against 56.3%/52.5%; rotation at 2.1% against 25% chance.

READ THIS LINE CAREFULLY

Five of six preregistered predictions were wrong. That is not a weakness of the project — it is the strongest single piece of evidence that the predictions were genuinely made in advance. A researcher who writes predictions after seeing results does not record a 1-in-6 hit rate.

8.4 One approach that failed and is preserved as a bound

The invariant-features line is worth separating out, because it is the clearest example of a result that is genuinely informative **and** genuinely negative.

1. **A supervised probe showed the information is there.** Permutation-invariant distributional summaries separate all four fault modes near-perfectly (the confusable trio at 0.987 on T11, 0.998 on T5). That withdrew W05 and looked like a clear route forward.
2. **Building the detector that recommendation implied made things worse.** The one-class invariant detector scored 0.617/0.680 against the guard's 0.672/0.742, and 98.5% of healthy episodes trended — against a 31% best elsewhere. The trend survives unclipping, so it is not a scoring artefact.
3. **The other half of the recommendation failed too.** Label-free attribution on the same features: 51.1%/39.7% against 56.3%/52.5%, with rotation named correctly **2.1%** of the time against 25% chance.

4. **The diagnosis is specific.** `sd` is a generic spread measure that every fault elevates, so it swamps the specific features — which is the `profile` residual problem again, in new clothes. The preregistration for that study explicitly declined a specificity ordering, on reasoning the result showed to be wrong.

WHY THIS FAILURE IS WORTH MORE THAN THE SUCCESS WOULD HAVE BEEN

The general lesson, and it is a real methodological finding: **a supervised demonstration that information exists has now twice failed to translate into a label-free rule that uses it.** An upper bound computed with labels is not a design for a monitor that never sees them. Claim E04 was **narrowed in place** to say so, with a (!) marker, rather than being quietly deleted.

SECTION 9

9. Results, reconstructed from the data

Every number in this section was regenerated during the audit by running `scripts/31_verify_claims.py`, which recomputes each headline from the stored data files rather than reading it from a report. All 93 checks matched.

9.1 Phase 1–2: the preregistered negative result

Test	n	τ	p	Verdict
Primary — robust dispersion, pre-transition	21	+0.743	0.0002	Significant
Parallel — trial-to-trial dispersion	21	−0.419	0.0068	Significant, opposite sign
Sensitivity — session level	11	+0.745	0.0012	Significant
Limitation check — healthy baseline only	—	+0.857	0.0018	This is what overturns it

The indicator rose *more steeply during healthy performance* ($\tau = +0.857$) than it did overall (+0.743). The limitation check was written into the frozen design in advance. Figure: `reports/figures/11_t11_why_negative.png`.

What the indicator was actually measuring — three converging lines

- **Firing rate.** Mean firing rate falls **56.5%** across T11's 142-day record. A linear fit of the indicator on it gives $R^2 = 0.707$. Controlling for firing rate, the indicator's link to performance goes non-significant ($p = 0.858 \rightarrow 0.260$, $p = 0.17$). Controlling for elapsed time, essentially nothing remains. And mean firing rate alone predicts performance ($p = -0.880$) **as well as the full pipeline does**.
- **Task invariance.** On extra T11 sessions using completely different tasks — including free web browsing — the indicator differs from the same-day cursor value by only **5.9–8.4%**, against a threefold range across the record. It measures **the recording**, not the human–decoder system.
- **Participant disagreement.** In T5 the indicator **falls** during degradation, so its reversibility test passes vacuously. No cross-participant generality was demonstrated.

THE MOST IMPORTANT SENTENCE IN THE PHASE 1–2 REPORT

The design flaw, stated plainly in the project's own report: the frozen design **did not require the block-level series to be de-trended before testing it for a trend**. The synthetic controls had already established that de-trending is necessary — that is precisely why the detector correctly stayed silent on monotonic drift in simulation. Its omission is why recording drift produced $p = 0.0002$. **This was not corrected retrospectively**. Doing so after seeing the outcome would have destroyed the preregistration. It defined Phase 3 instead.

9.2 Phase 3: the benchmark

Of 48 configurations	Count	Share
Ran	48	—
Found any operating point at all	47	98%
Achieved a positive median lead time	10	21%
Met the false-alarm budget ($\leq 0.1/h$)	3	6%
Passed the silence gate ($\leq 10\%$ of healthy episodes trend)	0	0%
Passed all five gates	0	0%

The grid is 4 detectors $\times$ 4 transforms $\times$ 2 baseline strategies on T11 (32 configurations) plus 4 detectors $\times$ 4 transforms $\times$ 1 baseline on T5 (16), for 48 in total — T5 was never run with the calibrate-once baseline. Full table in `reports/BENCHMARK_SUMMARY.md`.

WHY NOTHING PASSES

The binding constraint is not detection — it is specificity. Several configurations do warn before performance falls. But the risk signal is never quiet during healthy operation, so any threshold low enough to catch a fault early also fires constantly on healthy record. On the best T11 configuration the operating point is 50.46, giving **3.41 false alarms per hour against a 0.1/h budget** — missed by 34×.

9.3 But the information is present

Condition	decoder_guard AUC	Reading
Calibrated once, applied months later	0.491	Chance. Calibrate-once carries literally no information. This is a bound, not a shortfall of effort.
Re-baselined on recent healthy data, T11	0.693 (window) / 0.672 (episode)	Real information
Re-baselined on recent healthy data, T5	0.707 (window) / 0.742 (episode)	Real information, replicated

9.4 The monitor beats the trivial comparator, and exactly where it was designed to

Phase 1–2 found that counting activity matched a five-dimensional pipeline. This is therefore the comparison that decides whether any of the engineering was worth doing.

Fault mode	decoder-guard (T11 / T5)	Counting activity (T11 / T5)	Verdict
Overall signal loss	0.61 / 0.64	0.80 / 0.78	Counting wins — correct , this fault is nothing but overall activity
Electrodes dying	0.65 / 0.75	0.73 / 0.77	Counting wins narrowly
Channels drifting apart	0.79 / 0.73	0.41 / 0.40	Counting is at or below chance — as constructed
Signal shape rotating	0.76 / 0.71	0.52 / 0.51	Counting is at chance — as constructed

Counting activity is at or below chance on both faults that conserve total activity, while the monitor reaches 0.71–0.79. **The pattern replicates on both participants**, who disagreed in Phase 1–2.

9.5 The clean positive: task changes are not mistaken for faults

The dataset contains days on which the same participant used the cursor task *and* did something entirely different — including free web browsing — through the same electrodes, with nothing wrong.

Day 658	Same task	Different task	Real fault	Separation
decoder-guard	11.16	8.67	135.35	15.6×
Robust dispersion (the Phase 1–2 indicator)	0.14	0.46	0.29	Would false-alarm

Day 665 gives 70× separation. Robust dispersion fails in the worst possible direction — a *healthy task change* scored higher than a *real fault*. A monitor that alarms when someone switches application trains its user to ignore it. **Limitation, stated in the report:** this rests on two days and four blocks. It is a demonstration, not an estimate, and no confidence interval is quoted because none would be meaningful.

9.6 Why nothing can pass: the arithmetic, then the cause

The arithmetic of the operating point

	T11	T5
Healthy windows in the test split	17,014	17,337
Healthy hours	23.6	24.1
Alarms the 0.1/h budget permits	2.4	2.4
Required per-window false-positive rate	1.4e-04	1.4e-04
Observed per-window AUC	0.693	0.707
Detection achievable at that operating point	0.18%	0.03%
AUC needed for 80% detection there	0.9992	0.9992

An AUC of 0.999 is not a detector that needs tuning — it is a different measurement problem. The budget as applied demands near-perfect discrimination on every one of **720 decisions an hour**.

The mis-specification, and what it does and does not license

A CORRECTION THAT HAD TO BE MADE CAREFULLY

0.1 false alarms per hour was set as a usability requirement, and as a requirement on an **alarm** it is sensible. The error was applying it to a system that re-decides every 5 seconds, which quietly converts a mild usability constraint into a demand for 0.9992 AUC. A deployed monitor need not re-decide every 5 seconds: “should this session be flagged for a recalibration check?” is a **once-per-session** question. **What this does not license:** relaxing a target after failing to meet it is the classic way to manufacture a success. The argument is *not* that 0.1/h was too strict — the per-hour figure is unchanged. It is that a per-hour alarm budget and a per-5-second decision rate are different quantities, and the design conflated them. **The honest headline stays: at the operating point the design specified, no configuration works.**

The same detector, judged once per session	T11	T5
Session-level AUC	0.673	0.742
Detection at a 10% false-flag rate	14.4%	30.4%
Detection at a 5% false-flag rate	8.5%	17.0%
AUC needed for 80% detection at 10%	0.933	0.933

Aggregating to session level helps one participant and hurts the other (T5 0.707→0.742; T11 0.693→0.673), so it is **not** the rescue either — the participant disagreement shows up here too. But it moves the problem from impossible to merely hard: a specific gap of 0.67–0.74 achieved against ~0.93 required.

The cause: a session is one measurement, taken 55 times

	T11	T5
Windows per session	55	42
Lag-1 autocorrelation of the risk signal	0.995	0.980
Effective independent samples per session	0.1	0.4

This single fact explains every negative result in the project:

- **CUSUM cannot help** — accumulating 55 copies of one measurement adds nothing.
- **Longer or more windows cannot help** — there is no $\sqrt{N}$ to collect.

- **The silence gate always fails** — a series with $r = 0.995$ is a trend, so testing it for one will nearly always find one.
- **Six aggregators change nothing** — none beats a plain median; the tail-leaning ones sit near chance.

9.7 The day effect: the finding nobody was looking for

Result	Value	Status
Monitor accuracy varies by session on T11	0.319 to 0.974	ESTABLISHED (C08)
Share of that spread that is sampling noise	25.5% ($I^2 = 0.856$)	ESTABLISHED
Session accuracy vs that session's absolute decoder error	$\rho = -0.720$, $p = 0.0055$	ESTABLISHED (C09) — preregistered, direction committed in advance
Controlling for early-warning window length	-0.773	Strengthens
Controlling for achieved fault damage	-0.794	Strengthens
Controlling for trace noise	-0.333	Collapses — this is W02
A monitor's fit going stale over 142 days	Permutation $p = 0.128$ (naive test said 0.003)	ESTABLISHED (C07) — no measurable staleness
Cost of pooling episodes across sessions	0.003 AUC	ESTABLISHED (C13) — pooling is not diluting anything
The pattern is shared by distribution-based detectors, absent from scalar ones	guard vs distribution_shift: 0.835; vs mean_activity: -0.060	ESTABLISHED (C10) — it belongs to a class of monitor

WHAT THE DAY EFFECT DOES AND DOES NOT ESTABLISH

Ageing does not matter; which day you use the monitor on decides everything. That is a deployment-relevant finding, and it inverts the intuition the project started with. But it also carries its own retraction: **which property** of a session causes the swing is **withdrawn** (W02). Trace noise, decoder error, spurious crossings, task geometry and margin over chance are all mutually correlated, and the partials are identical in both directions — which is collinearity, not mediation. And whether the effect is failed *detection* or degraded *labels* is formally **UNANSWERABLE** with this corpus (U01): filtering to unambiguous faults leaves 7 sessions, where $|p|$ must reach ~ 0.79 to clear $p < 0.05$.

9.8 Calibration efficiency — the first clean, non-null positive

Calibration data	T11 AUC	T5 AUC	Note
10 windows (scattered)	—	—	Not a measurement. The profile covariance is singular below $n = K+1$ (minimum eigenvalue 0.000). It is also the highest T11 AUC, and would have supported a false headline that <i>more</i> calibration data makes the monitor worse (L04).
20 windows (scattered)	0.653	0.740	Minimum eigenvalue 0.057 — well posed
20 windows (contiguous)	0.665	—	Contiguous draw confirms the count translates into real minutes
40 windows (contiguous)	—	0.740	
The entire healthy record	0.648	0.739	About two minutes of healthy recording reaches what the whole record reaches, on both participants

Why L04 matters more than C06. The $n = 10$ point is the most attractive number in the table and it is not a measurement at all. A project that reported it would have published a striking, memorable, and entirely spurious finding. It was caught by checking the conditioning of the covariance matrix rather than by looking at the AUC.

9.9 Attribution: a capability that was built, scored, and nearly thrown away

Result	T11	T5	Chance
Overall attribution accuracy	56.3%	52.5%	25% / 33%
GAIN_DRIFT named correctly	99.3%	—	
GEOMETRY_ROTATION named correctly	0.0%	0.0%	

The rotation failure is fully diagnosed and the diagnosis retracts the preregistration. During rotation episodes profile *is* lit — in 80% of them — but dispersion sits at $z = 16.5$ against profile's 1.92, so a largest-wins rule picks dispersion 98% of the time. The cause is definitional: rotation is injected with **norm-preserving** Givens rotations, and dispersion is defined as channels spreading apart with the total conserved. **They are the same signature by definition.** Re-ordering or re-weighting the rule cannot fix it; the components would have to be redefined.

The register note on C14 is worth quoting: attribution was “implemented and scored since the guard was written, printed to stdout and **captured nowhere** until now.” A real capability existed for a week without being a result, because nothing wrote it to a file.

9.10 What the project ruled out, and what remains

Ruled out on evidence	The evidence
Better decision rules	Three compared by full curve; all 0% inside the budget
Longer or more windows	Effective $n = 0.1$ — no independent information to average
Better session-level aggregators	Six tried; none beats a median
Calibrate-once deployment	AUC 0.491 — literally no information
Better features alone	F1 and F2 <i>are</i> better features under a matched control (+0.106/+0.158 and +0.059/+0.095) and it still is not enough: <code>decoder_guard</code> beats its own features under that scorer by +0.102/+0.233, so its advantage is the four-component decomposition , not the per-channel means it consumes
A sharper one-class detector on invariant features	Built and measured: worse on both axes (C16)
Label-free invariant attribution	Built and measured: worse (C17)
Abstention	Four candidates, two stages, nothing passes stage 2 (C11)

What remains as the only untried direction: a fundamentally better *measurement*, combining better features **with** the four-component decomposition rather than substituting for it — which is a new study needing its own preregistration, not a change to make on the strength of a post-hoc table. And, ahead of that in value, **a third participant.**

SECTION 10

10. Reproducibility audit

Reproducibility here is not a claim; it is a gate that runs. This section reports what that gate covers, what it caught, and what it provably cannot see.

10.1 What the gate checks, and its state on re-run

Check	Rule	State at audit
Imports	Every third-party module imported anywhere appears in <code>requirements.txt</code>	PASS — 6 imports, all declared
Producers	Every data/processed file a script <i>reads</i> is <i>written</i> by some script	PASS — 35 consumed files, all with a producer
Order	No script reads a file produced by a higher-numbered script	PASS — no inversions, with two scripts documented as running last
References	Every script named in a report or the README exists	PASS — 70 documents scanned at the time of this audit. Minor finding: the checked-in <code>REPRODUCIBILITY_AUDIT.md</code> still says 53, the count when it was last written by hand around the generated output

10.2 What the gate caught on its first run

FINDING 1

A headline figure that no committed script produced. The combined sign test in `reports/WINDOW_OVERLAP.md` — 11 of 13 days improved, $p = 0.011$ — was computed in an ad-hoc shell one-liner, and `31_verify_claims.py` was checking a JSON file that **nothing in the repository regenerated**. A fresh clone would have failed that claim. **This is precisely the failure the audit exists to catch: a verified claim resting on an unreproducible file looks exactly like a verified claim.** The test now lives in `54_window_overlap.py` and produces the same numbers.

FINDING 2

The README's "numbered in dependency order" claim was false. `31_verify_claims.py` reads output from fifteen higher-numbered scripts, because it recomputes every headline from whatever the pipeline produced. Renumbering would break the script names quoted throughout the reports, so the README now states what is true: **two scripts run last**, the verifier and this audit.

10.3 The audit's own three bugs

Documented in the tool's own report, because a tool that reports its own artefacts as findings is worse than no tool. Its first run reported 27 problems, of which **1** was real.

Bug	Symptom	Fix
f-strings parsed by regex	<code>f"calibration_curve{sfx}.csv"</code> was captured as the filename <code>}.csv</code> , reported as an orphan read by ten scripts	Flatten <code>ast.JoinedStr</code> nodes in the AST
Filenames held in variables	<code>ckpt = OUT / f"staleness{sfx}.csv"</code> followed twenty lines later by <code>to_csv(ckpt)</code> looked like a read with no write	Follow one level of assignment — not full dataflow, but enough
The audit as its own producer	The phrase <code>-> write_text</code> inside its own allowlist matched the write-call pattern, making it the recorded producer of the decoder files	Exclude the audit from its own scan

10.4 The two files a static audit cannot resolve

`reference_decoder.npz` and `reference_decoder.json` are written by `18_reference_decoder.py` through a `decoder_paths()` helper that *returns* the path, so the filename literal never appears near the `np.savez / write_text` that writes it. Following that needs interprocedural analysis. Both are named explicitly in a two-item `RESOLVED_BY_HAND` allowlist with their real producer, **so the check stays a clean gate rather than a permanently-failing one. Anything not on that two-item list is a real finding.**

10.5 Reproducibility in practice: can a stranger run this?

Requirement	State	Evidence
Clone and run the verifier without the raw data	Yes	All 136 processed files are tracked since commit e421166. Before that commit the verifier could not run on a fresh clone — the outputs it checks were gitignored. Verified by actually cloning.
See the figures on GitHub	Yes	17 figures tracked since the same commit. Before it, every image link in every report was broken on GitHub.
Re-download the raw data	Yes, parameterised	<code>01_download_dataset.py</code> takes a DOI (commit 227570d), so a third participant needs credentials, not a rewrite.
Reproduce the raw data byte-for-byte	Yes, by checksum	SHA-256 recorded in the manifest and verified at download time.
Reproduce the fault corpus exactly	Yes	Master seed, per-episode seeds, and a SHA-256 of the episode list, all in the injection plan. Re-drawing is refused without a recorded reason.
Reproduce the environment exactly	Partly — gap	Six packages named with <code>>=</code> lower bounds, not exact pins. The audit's own report flags this. See roadmap R6.
Know which script produced which number	Yes	Every report carries a Reproduce: line naming its script; every register entry names its verifier checks.

10.6 What reproducibility here does not mean

THE HONEST CEILING ON SELF-CHECKING

Quoted from the project's own report, because the audit agrees with it and cannot improve on it: **"Static only.** It cannot detect a script that runs and produces wrong output, only one that cannot run at all for want of an input." The claim verifier has the matching limit: it checks that reported numbers match stored files, not that the computation behind those files is right. **Errors #1, #3 and #4 in Section 3.3 all passed every gate** and were caught by a human noticing that a number looked wrong.

SECTION 11

11. Scientific rigour audit

This section assesses the project against the practices that separate a result from an anecdote. It is organised as a scorecard, then the contradictions and open methodological problems that the scorecard does not capture.

11.1 Rigour scorecard

Practice	Assessment	Evidence
Preregistration	Exceptional	Twelve separate frozen designs or preregistration notes. Each names its criterion and, in six cases, a prediction — before the analysis. The Phase 1–2 freeze carries a git commit hash and SHA-256 of its inputs.
Ground truth	Exceptional	Constructed rather than assumed. Onsets drawn and locked under checksum before any detector existed , with the grading system committed before the monitor.
Controls	Strong	Synthetic positive/negative controls with known answers; a measured chance level by shuffled pairing (not assumed 90°); a calibrate-once arm that correctly lands at 0.491; a 12-random-channel control for the invariant-feature result; NONE episodes throughout the corpus.
Blinding / leakage control	Strong	The detector contract forbids access to decoder output, task, performance and labels. One-class fitting on healthy data only. Splits by block. Test set read once.
Replication	Adequate, and honestly bounded	Every headline is reported on both participants. The comparator result replicates. But n = 2 and they disagree — the project names this as its binding limitation rather than burying it.
Correction of errors	Exceptional	Fourteen documented errors, three of which changed a conclusion. Corrections appended inline rather than replacing the original text. Five claims formally withdrawn.
Statistical practice	Good, after a serious lapse	The unit-of-analysis error was real and affected every p-value in one report. It was found, measured (26.6×), corrected, published as a correction, and then encoded as a linter so the class cannot recur silently.
Negative results	Exceptional	The project's headline is a negative result, reported as such, with its cause located to three decimal places. Nulls are published in full: the feature study, the combination study, abstention, the invariant detector, the invariant attribution.
Reasoning recorded	Exceptional	A 4,050-line research log with 54 dated headings, touched by 58 of 123 commits. Commit messages state what was invalidated, not just what changed.
Unit tests	Absent — gap	No test suite. Correctness rests on output-level gates and synthetic controls. See Section 6.5.
External review	Absent — gap	Audit judgement. Every check in this project was written by the same author as the work it checks. That is a structural limit no amount of internal discipline removes.

11.2 Contradictions and tensions found

Flagged as required, including ones the project has already flagged itself.

#	Tension	Status
1	C02's headline number is window-level while the project's own C04 says windows are not independent.	Resolved by explicit labelling, not by changing the number. The register entry for C02 now carries a note: the 0.693 is window-level, the episode-level equivalents are 0.672 and 0.742, and no interval or p-value may be built on the window-level figure . Both numbers are verified separately.

#	Tension	Status
2	The 0.1/h budget is called both “the specified operating point” and “mis-specified”.	Genuine tension, handled carefully. The report keeps the original headline (“at the operating point the design specified, no configuration works”) and explains why a per-session decision rate is a different quantity. It explicitly refuses to treat this as licence to relax the target.
3	E04 says the attribution failure is “a design problem, not missing information”; C17 says every label-free attempt to use that information is worse.	Resolved by narrowing E04 in place , with a (!) marker: it is evidence the information exists, not a design for a monitor. A supervised upper bound is not a capability.
4	T11 and T5 disagree in sign on the strongest cross-participant relationship in the project.	Unresolved, and correctly reported as such. Claim C12: mean output speed tracks decoder error on both participants and clears the threshold on both — with opposite signs (+0.681 vs −0.943). Any rule tuned on one array runs backwards on the other.
5	The day effect is established on T11 but T5 cannot corroborate it.	Reported, with the reason. On T5 the session spread tracks the fault mix each session happened to draw ($p = 0.886$); on T11 it does not (0.050). So T5 is confounded for this question. This was found only because a comparison bug was fixed — error #1.
6	The severity ladder is used throughout but is only valid 57–67% of the time per cell.	Documented as L01, not resolved. Severity is valid in aggregate only, and its validity degrades with baseline error. Every result stratified by severity inherits this.

11.3 Methodological problems that remain open

Problem	Why it is not closed	Consequence for the claims
n = 2 participants, who disagree	Only two suitable public records were obtained. A third (T15, Card et al. 2024) is identified and costed but blocked on Dryad credentials.	The binding limitation. With $n = 2$ an improvement cannot be distinguished from noise, and no cross-participant generality is demonstrated.
Open-loop evaluation	A direct consequence of Amendment 1. An injected episode is a recording and cannot adapt.	Performance is decoder output error against recorded intended direction, not task success. The compensation gap can be reasoned about but not newly created.
The corpus is blind where it matters most	On near-chance sessions a fault does almost no measurable damage.	6 of 13 T11 sessions cannot be scored on unambiguous faults at all. The sessions where the monitor performs worst are the sessions whose labels are weakest — and disentangling those two is formally UNANSWERABLE here (U01).
Injected faults may not resemble real degradation	Every fault is a mathematical operation on a recorded array. Nobody has checked whether real hardware degradation resembles $X * (1 - \text{severity} * \text{ramp})$.	The circularity risk the project names itself: if the monitor only ever meets degradations the researcher wrote, passing partly measures the researcher’s imagination. This is what Section 13 is designed to attack.
Attribution names only 3 of 4 components	GEOMETRY_ROTATION is definitionally confounded with dispersion.	Attribution partly collapses onto whatever is chronically lit. Fixing it requires redefining the components, not re-weighting the rule.
Every check was written by the author of the work	No external reviewer, no adversarial collaborator.	Audit judgement: this is the largest remaining structural risk, and it is not fixable by writing a sixth gate. It is fixable by a mentor, a teacher, or a judge reading the code.

11.4 What a hostile reviewer would attack, and whether it holds

Attack	Does it land?
“You found a correlation and called it a detector.”	No. The project’s headline is that its detector fails the gates, and the gates were fixed in advance. There is no correlation being sold as a capability.

Attack	Does it land?
“You defined the event after seeing the data.”	No. Six deterioration definitions were compared using performance data only, one frozen in advance with input checksums, and the whole circularity problem is the project’s stated reason for building a corpus with known onset.
“You tried features until one worked.”	No. Four families named, list closed, criterion frozen with a checksum, and the one family that <i>did</i> clear the bar on T11 was reported as a null because it failed on T5.
“Your p-values are wrong.”	They were — and the project found it, measured it at 26.6×, corrected it inline, and built a linter. This attack has already been made and answered from inside.
“Your faults are made up, so your result is about your own imagination.”	This one lands , and the project agrees. It is the single strongest motivation for the physical experiment in Section 13.
“n = 2 is not a study.”	This one lands. The project names it as the binding limitation and ranks a third participant above any detector improvement. It is blocked on credentials, not on effort.
“An AI wrote this.”	Partly, and it is disclosable. The commits are AI-authored under the researcher’s direction. What the researcher must be able to do is explain the method, the errors, and why each correction was made — all of which are documented. Section 12 flags the missing piece: a written AI-use disclosure.

SECTION 12

12. ISEF readiness

The researcher deferred ISEF paperwork twice. This section is therefore an **assessment of readiness**, not paperwork, and it separates what is ready now from what still requires a decision only the researcher can make.

12.1 Rules and safety

Requirement	Status	Basis
Human participants	None — Form 4 not expected	Amendment 1 (26 Aug 2026) removed every configuration involving a person, including the researcher. Data collected from oneself would still be human-participant research and is excluded. Commit 7102ff6 states plainly that “participant” never means a recruited person.
Vertebrate animals	None	No animal work of any kind
Hazardous biological agents	None	None
Hazardous chemicals / devices	None so far	If the rig in Section 13 is built, it is low-voltage bench electronics connected to no person. The ISEF plan already flags this: confirm rather than assume that no Form 3 is needed.
Human data	Public, CC0, de-identified	The MINDFUL deposit is CC0-1.0 and was collected by other researchers under their own approvals. Analysis of pre-existing de-identified public data is the standard exempt case, but the SRC should confirm it rather than the researcher assuming it.
Rules Wizard	Must be re-run	The plan says so explicitly, once the hardware decision is made.
AI-use disclosure	GAP — not written	The plan carries a §C-AI placeholder and warns that AI-disclosure wording has moved year to year, and that AI use is one of three things the form lists as grounds for disqualification. The evidence to write an excellent disclosure exists (Section 3); the text does not.

12.2 Research plan components

Component	State	Where it lives
Rationale	Ready	ISEF_RESEARCH_PLAN.md §A, derived from PROJECT_DEFINITION.md; preliminary findings added in commit df4a056
Research question / engineering goal / hypotheses	Ready	Kept as three separate statements — see Section 4.3
Variables	Ready	Independent, dependent, controlled and constant all specified — see Section 4.4
Procedures	Ready	research/procedures.md — 40 numbered procedures, described as methods-section ready
Risk and safety	Ready for the computational project; needs a paragraph if the rig is built	Plan §C
Data analysis	Ready and unusually strong	Frozen designs, five gates, a claims register, four automated audits
Bibliography	Present — with one unsupported claim	10 formal citations in ISEF_RESEARCH_PLAN.md §D, each with a DOI — Barrese 2013, Helmich 2024, Hughes 2021, Karpowicz 2025, Maturana 2020, Pun 2024, Scheffer 2009, Sponheim 2021, van der Bolt 2021, Wilkat 2019. Listed in full in Appendix B. But the section header claims “the full annotated review contains 25 and is in the project repository” — and it is not. See 12.3.

Component	State	Where it lives
Title	Drafted, two options	“Decoder-Guard: An Interpretable Real-Time Health Monitor for Brain-Computer Interface Decoders, Validated Against Degradation of Known Onset”, or the shorter “Seeing Failure Before It Shows”
Researcher-supplied fields	Blank by design	Name, school, adult sponsor are marked [[RESEARCHER]] — the plan refuses to invent them

12.3 One document the project depends on and does not contain

THE LITERATURE REVIEW IS CITED BUT ABSENT

Audit finding, and the most significant omission found in the whole repository. A “literature review” is referenced as an existing, decision-shaping document in at least six places — `design_decisions.md` quotes it directly (“It’s just MINDFUL with different statistics”), `PHASE1_2_REPORT.md` records a correction to it (it had the two participants reversed), the research log cites its framing (C) as the origin of the degrading-plant-plus-compensator hypothesis, and `ISEF_RESEARCH_PLAN.md` §D states that it “contains 25 [sources] and is in the project repository.” **It is not in the repository, and no commit in the entire history ever added it.** Verified with `git log --all --diff-filter=A --name-only`.

Two consequences, and they are different in kind. **Scientifically**, a reader cannot check the reasoning that led to the project’s framing, because the document that carries it is external to the artefact — this is the largest **UNRECOVERABLE** item in the audit. **For competition**, the ISEF plan makes a factual statement about the repository’s contents that the repository does not support, and a judge who looks will find that out. The fix is one of two things, and either is fine: commit the review, or amend the sentence to say where it actually lives. What is not fine is leaving the claim as it stands.

12.4 How this presents to judges

Judging dimension	Assessment
Originality	High. The corpus of 1,850 fault episodes with known onset does not exist publicly, and the five-gate battery is a genuinely missing methodological product. The framing — “you cannot validate an early-warning system without controlling onset” — is a real argument, not a project slogan.
Scientific thought	Very high. Five withdrawn claims, five of six preregistered predictions wrong and reported, an unanswerable question labelled as unanswerable rather than answered no. This is the dimension where the project is strongest.
Thoroughness	Very high. 123 commits, 65 scripts, 45 reports, 4,050 lines of log, 93 automatically verified claims.
Skill	High, with a caveat. The analysis is sophisticated. The caveat is that the code was AI-authored, so the researcher must be able to explain the method and the errors in their own words. The material to do that is all in the log.
Clarity	Good, and improving. Two rendered HTML write-ups exist, one published as a shareable artefact. But there are 45 Markdown reports and a judge will read at most three. See R2.
The hardest question to answer	“What did you do, as opposed to the assistant?” This is not a trap, and it has a good answer: the researcher set the constraints that shaped the entire project — no human participants, do not optimise for significance, do not select a deterioration definition after seeing results, record everything including the mistakes. Those constraints are why the project has a claims register and a linter instead of a fragile positive result. That answer needs to be rehearsed, and it needs the AI disclosure in 12.1 written.

12.5 The presentation risk nobody has addressed yet

AUDIT JUDGEMENT ON PRESENTATION

The project's headline is a **negative result**, and negative results are harder to present than positive ones — not because they are worth less, but because the audience must be walked to the point where they see why the negative is informative. The chain that has to land in about ninety seconds is: **(1)** decoders silently fail; **(2)** you cannot validate a warning system without knowing when the thing being warned about started; **(3)** so I built 1,850 faults with known onset; **(4)** no configuration passes; **(5)** and here is the single number that explains why — a session contains one measurement taken 55 times. Every one of those five steps is documented. **What does not yet exist is the ninety-second version.**

SECTION 13

13. The original data-collection component, derived from the findings

Status: DESIGN ONLY. Nothing has been built and nothing has been measured. No number in this section is a result. The design is recorded in `research/ORIGINAL_DATA_COLLECTION_DESIGN.md`, committed 5 September 2026.

13.1 The chain, stated in the required form

WHY A PHYSICAL EXPERIMENT, IN ONE SENTENCE

The physical experiment is being proposed because the current research found that no configuration of a decoder-health monitor passes the five gates on real intracortical data, and that the reason is a single measured property — a session contains roughly one independent measurement (lag-1 $r = 0.995$, effective $n = 0.1$), **which suggests** that the failure may be a general property of any multichannel sensor array that drifts slowly, rather than something specific to cortex — and that the fault corpus, every episode of which is a mathematical operation on a recorded array, may not resemble real physical degradation at all, **but the existing data cannot determine** either of these, because nobody controlled anything in the recordings, only 13 usable sessions exist, five session-level variables are mutually entangled, and no real hardware fault was ever observed. **Therefore the next experiment will measure** detection AUC, silence-gate pass rate, lag-1 autocorrelation and severity-ladder validity on a physical multichannel optical sensor array, **under conditions of** physically realised faults of the same four types at independently manipulated baseline SNR and task geometry — plus a parallel arm in which nothing is injected and the rig is simply left to degrade — **in order to determine** whether the monitor's failure is neural-specific or array-general, and whether simulated fault injection resembles real degradation.

13.2 The mandatory findings → experiment table

Every row begins with a finding that exists in this repository, names the file that establishes it, states what the archived data cannot resolve, and gives the measurement that would resolve it. **A row with no repository finding behind it does not appear.**

Finding in this project	Established in	What archived data cannot determine	What the experiment measures	Prediction, to be preregistered before building
Lag-1 $r = 0.995$ within a session; effective $n = 0.1$. This single number explains why CUSUM cannot help, why longer windows cannot help, and why the silence gate always fails.	AGGREGATION_LIMIT C04	Whether 0.995 is a fact about cortex or about any slowly-drifting multichannel array . There is no second kind of array in the dataset.	Lag-1 autocorrelation of the risk signal within a rig session, at matched window and step geometry.	Lag-1 $r \geq 0.99$ on the rig. If it holds, the negative result generalises to sensor-array health monitoring; if it does not, the failure is neural-specific.
0 of 48 configurations pass the silence gate. The binding constraint is specificity, not detection.	BENCHMARK_SUMMARY C03	Whether a monitor could pass if the array's drift were slower, or if sessions were plentiful. 13 usable T11 sessions is the ceiling and it cannot be raised.	Silence-gate pass rate across hundreds of rig sessions — one T11-equivalent session takes about 20 minutes and the rig can run unattended.	The silence gate still fails. Session count was never the binding constraint; serial correlation was.

Finding in this project	Established in	What archived data cannot determine	What the experiment measures	Prediction, to be preregistered before building
Five session-level variables are mutually entangled — decoder error, trace noise, spurious crossings, task geometry, margin over chance. Partial correlations are identical in both directions, which is collinearity, not mediation.	WHAT_DECODER_ERROR_MEANS W02, E01	Which one causes the accuracy swing. Separating them requires sessions where one varies while the others are held fixed. Nobody controlled anything in the original recordings.	Detection AUC with baseline SNR and task geometry independently manipulated , each held fixed while the other varies.	Baseline SNR dominates. If task geometry dominates instead, the P5 result (C09) is about the task, not the signal.
Every injected fault is a mathematical operation on a recorded array. GAIN_DRIFT had to be fixed twice before it was even mean-conserving.	The injector; PROJECT_DEFINITION §5.4	Whether real hardware degradation resembles $X * (1 - \text{severity} * \text{ramp})$ at all. No real fault was ever observed — this is the circularity risk the project names against itself.	The undesigned arm: physically realised faults whose feature-level signature emerges from optics and physics rather than from an equation, with onset known because the experimenter caused it.	Designed and undesigned faults are reported separately . Generalisation from the first to the second is a testable claim, not an assumption.
Angular error saturates near chance, and its saturation point moves per session (each day's own chance ranges 47.7°–102.7°). 6 of 13 T11 sessions cannot be scored on unambiguous faults at all.	UNAMBIGUOUS_EPISODES (L03); WHAT_DECODER_ERROR_MEANS	Whether a metric with more dynamic range at the bad end fixes it. The recommendation was published and could not be tested — the data has no sessions at controlled SNR.	Baseline SNR swept continuously across ~20 levels, ~200 short sessions in an afternoon; measure where angular error saturates and compare candidate replacement metrics.	Angular error saturates well before the signal is exhausted. A replacement metric recovers measurable range on sessions the current one cannot score.
Severity-ladder validity degrades as baseline error rises — monotone in only 57% (T11) and 67% (T5) of session×mode cells, $\rho = -0.677$ and -0.926 . This is the only relationship that replicates with the same sign on both arrays.	SEVERITY_LADDER_VALIDITY L01	Whether the degradation is a property of the ladder or of the sessions it was applied to. Both vary together and neither can be held fixed.	Ladder monotonicity across the SNR sweep, with severity levels identical throughout.	Monotonicity degrades with baseline error on the rig too, at a comparable slope.
A monitor's fit does not go stale over 142 days (permutation $p = 0.128$ against a naive 0.003) — but which day you use it on decides everything (AUC 0.32–0.97, $I^2 = 0.856$).	STALENESS_AND_DAY_VARIANCE C07, C08	Whether natural, uninjected drift produces the same statistical signature as any of the four injected modes. The archived record has natural drift but no controlled comparison.	The longitudinal arm: one session daily for 6–10 weeks with nothing injected, logging temperature and humidity as covariates. Dust, LED aging, thermal cycling and connector oxidation are allowed to happen.	Natural drift matches RATE_LOSS or GAIN_DRIFT most closely. If it matches none , that is a significant finding about how fault benchmarks are built — including this one.

13.3 The apparatus

One rig, built once, runs three studies. An N-channel optical sensor array observes a moving 2D stimulus; a ridge decoder predicts stimulus direction from the array response. That is **structurally identical** to predicting intended cursor direction from firing rates, which is what makes the existing pipeline apply unchanged.

Element	Implementation	Cost
Stimulus	WS2812 addressable LED matrix producing a known 2D direction each frame	~\$15
Sensor	Raspberry Pi camera module; N virtual channels are fixed image regions, and frame-averaged region intensity is the “firing rate”	~\$25
Compute	Raspberry Pi 4 or 5	~\$60

Element	Implementation	Cost
Rotation	28BYJ-48 stepper and driver, rotating the camera relative to the stimulus	~\$5
Attenuation	Neutral-density filter sheet	~\$8
Covariates	DHT22 temperature and humidity sensor (longitudinal arm only)	~\$5
Enclosure	Cardboard or printed mount; ambient light controlled	—
Total	No soldering strictly required	≈ \$115

How each injected mode becomes a physical event

Corpus mode	Physical counterpart	Onset known?	Signature designed?
CHANNEL_DROPOUT	Occlude specific image regions	Yes	Yes
RATE_LOSS	Neutral-density filter over the whole stimulus	Yes	Yes
GAIN_DRIFT	Per-region illumination change	Yes	Yes
GEOMETRY_ROTATION	Literally rotate the camera on the stepper	Yes	Yes
Undesigned faults	Loosen a connector; warm a component for thermal drift; introduce EMI from a nearby device; let contact impedance drift over hours	Yes — the experimenter did it and logged it	No — it emerges from physics

Known onset with an undesigned signature is the strongest ground truth available to this project, and it requires no participants at all. It is the direct answer to “your faults are made up.”

13.4 Variables for the rig study

Class	Variable	Levels
Manipulated	Fault mode	4, physically realised, plus an undesigned arm and a no-fault control
	Severity	3, matched to the corpus definitions
	Onset time	Randomised and logged before the run, as in the corpus
	Baseline SNR	Swept independently — the manipulation archived data cannot offer
	Task geometry	Swept independently — likewise
Controlled	Channel count	Exactly 384 (matching T11) or 192 (matching T5) — an open decision
	Bin width, window geometry	Identical to the neural pipeline
	Decoder	Ridge regression, frozen after fitting, as in <code>18_reference_decoder.py</code>
Constant	Ambient light	Enclosure
	Stimulus speed distribution, frame rate	Fixed
Dependent	Angular error, session-level AUC, silence-gate pass rate, lag-1 autocorrelation, achieved damage	Unchanged from the neural study — that is the point

WHY THIS DESIGN IS CHEAP IN THE WAY THAT MATTERS

The analysis code does not change. The injector, the reference decoder, the harness, the five gates, decoder-guard and the claim verifier all apply unmodified. **The analysis instrument is already written and already validated by 93 checks** — which means the rig’s results are comparable to the neural results by construction rather than by argument.

13.5 The rule that governs what may be simulated

STATED VERBATIM IN PROJECT_DEFINITION.MD §5.5

The perturbation may be simulated. The response to it may not.

Legitimately simulated	Must be real
The degradation schedule — which mode, when, how fast	The signal's own non-stationarity. Simulating it assumes the answer.
Surrogate and permutation null distributions	The human's adaptive compensation (in the archived arm)
Power analysis and the severity ladder	The decoder's adaptive normalisation
The sensor-failure mechanism on the rig (switched attenuation)	The analog path it passes through

The reason this line matters: simulating the thing you are trying to detect is circular — you would be measuring your own generative assumptions. That is the same error, in a different costume, as choosing a deterioration definition after seeing which one gives the best result. **This project already refused that once.**

13.6 Designs considered and rejected

Rejected	Reason
EMG, EEG, OpenBCI, Muse — on anyone, including the researcher	Human participant research. Excluded by Amendment 1 and would require SRC/IRB review. Data collected from oneself still counts.
Camera-tracked hand movement as a “BCI-like” task	Same — still human participant research.
More neuron simulation in software	Not data collection. Redundant: a simulation layer already exists and adding another closes none of the four gaps.
Accelerometer vibration monitoring of a motor	Standard predictive maintenance, researched since the 1980s, and connected to none of this project's specific findings.
Robot arm performing a reaching task	Plausible but muddies the decoder/monitor framing, ~10× the cost, and addresses no documented gap.

13.7 Ranking, risks, and what must be decided first

Rank	Study	Verdict
1	Controlled degradation of a physical array	Best combination of novelty, feasibility and rigour. Attacks the worst gap, reuses 65 verified scripts, ~\$115, two weekends to build, no participant paperwork.
2	Longitudinal natural degradation	Highest novelty here , gated on 6+ weeks of calendar time. Costs nothing extra to run in parallel on the same hardware. Start on day one, harvest at the end.
3	Metric saturation sweep	Nearly free as a sub-study of #1. Not a standalone project.

Risks, stated plainly

- **Skill gap.** Hardware is a different skill from running scripts, and **a half-built rig producing noisy data is worse than no rig.** Mitigation: build the simplest version first — camera, LED matrix, no motor, 16 channels — get it producing data end to end, then scale to 384 channels and add rotation.
- **It is not neural.** This is the real weakness and it must be defended as a claim about **a class of signal**, not as a brain simulation. The claim under test was always stated at the level of multichannel arrays with slow drift; no claim is made that a photodiode resembles a neuron.

- **Timeline.** The longitudinal arm needs 6+ weeks of wall-clock time and is not compressible.
- **Nothing is built until a preregistration exists** stating the predictions derived from the neural findings before a single measurement is taken — the same discipline every study in this project has followed. The predictions in the last column of §13.2 are the draft of that document.

OPEN DECISIONS

Three decisions belong to the researcher and block the build: **(1)** the deadline — it decides whether the 6-week longitudinal arm is in or out; **(2)** whether to build hardware at all — scientifically the right call, and also real work, with a defensible computational project remaining if the answer is no; **(3)** channel count — 384 to match T11, 192 to match T5, or both.

13.8 What the integrated project becomes

Part	Source	Role
I. Archived	MINDFUL T11 + T5	The real-world instance. Establishes the failure, measures its cause, generates the predictions.
II. Original	The sensor rig	The controlled instance. Manipulates what cortex will not allow. Tests those predictions.
III. Computational	The existing pipeline	The shared instrument. Identical analysis on both, by construction.

THE REASON THIS DESIGN IS WORTH THE WORK

The structure that makes this unusual: **a computational finding is used to make falsifiable point predictions about a physical system, which is then built.** That converts the project's largest weakness — two participants and control over nothing — into its methodological centrepiece. It is also the only version of the original-data component that answers the sharpest criticism the project currently faces.

SECTION 14

14. Where the project stands, and what to do next

14.1 Current state, in one table

Dimension	State
Scientific result	Complete and negative, with the cause located. 0 of 48 configurations pass; the binding constraint is specificity; the cause is lag-1 $r = 0.995$. Achieved session-level AUC 0.673 (T11) and 0.742 (T5), against ≈ 0.99 required at the operating point the design specified and ≈ 0.93 required for 80% detection at a 10% false-flag rate.
Original artefacts	Three that did not previously exist: a corpus of 1,850 fault episodes with known onset; a five-gate validation battery whose first gate would have caught the project's own Phase 1–2 error; and a replicated demonstration that the monitor detects what counting spikes cannot.
Clean positives	Two. Task changes are separated from faults by 15×–70×. Commissioning costs about two minutes of healthy recording, on both participants.
Verification	93 claims recomputed from data files, all matching. Five automated gates, all passing.
Documentation	4,050-line log with 54 dated headings; 45 reports; 12 preregistrations; a claims register with 38 entries and 5 statuses.
Binding limitation	n = 2 participants, and they disagree. Blocked on Dryad credentials for a third.
Original data collection	Designed, not built. Three open decisions belong to the researcher.
ISEF paperwork	Deferred by the researcher, twice. The plan exists as a draft with researcher-only fields left blank.

14.2 Prioritised roadmap

Ordered by value per unit of effort, with the reason each sits where it does. Items R1–R3 are scientific; R4–R8 are hygiene that a judge will notice.

#	Action	Why here	Effort	Blocked on
R1	A third participant — Card et al. 2024 (T15)	Worth more than any detector improvement. With two participants who disagree, no generality is demonstrable. And it is cheap: even without a decoder, the silence gate needs only healthy recordings — enough to establish whether the failure belongs to these two arrays or to the approach. DOI 10.5061/dryad.dncjsxm85, 256 electrodes, different task, 11.6 GB. The downloader is already parameterised by DOI.	Low, once unblocked	Dryad credentials — DRYAD_CLIENT_ID and DRYAD_CLIENT_SECRET set in the environment config, not in chat
R2	Answer the three open hardware decisions, then preregister the rig study	Section 13 is a complete design that cannot start. The deadline decides whether the 6-week longitudinal arm exists at all, and that arm is the highest-novelty part.	A conversation, then a document	The researcher — deadline, whether to build at all, channel count
R3	Re-run the corpus with the window-overlap fix	L02 measures the bias at 0.007–0.012 AUC and shows it is conservative, so nothing changes — but the fix is one line (<code>start + window <= onset_bin</code>) and it removes a standing caveat from every number in the project.	Low — one condition, one re-run	—
R4	Write the AI-use disclosure	The highest-risk gap in the whole project. AI use is one of three things the ISEF form names as grounds for disqualification, and the plan carries only a placeholder. The evidence for an unusually strong disclosure already exists in Section 3 — fourteen documented errors caught and corrected.	Low	The 2026–27 form wording; confirm with the SRC

#	Action	Why here	Effort	Blocked on
R5	Commit the literature review, or amend the claim that it is here	A factual statement in the ISEF plan that the repository does not support. The review is cited and quoted in at least six files and is the origin of the project's framing, its threat model and its comparator choice — and no commit ever added it. The bibliography itself is fine: 10 formal citations with DOIs exist in ISEF_RESEARCH_PLAN.md. Either commit the review or say where it lives.	Low	—
R6	Pin exact package versions	The six dependencies use $\geq$ lower bounds. The reproducibility gate reports this limit itself. A pip freeze lockfile closes it.	Trivial	—
R7	Tag the pivotal commits	Nothing is tagged — not the design freeze, not either phase closeout. Tags make the preregistration story visible from one command instead of from a JSON field.	Trivial	—
R8	Write the ninety-second version	The project's five-step argument is fully documented across 45 reports and a 4,050-line log. A judge will read three of them. The compressed version does not yet exist in any file.	Low, and high-value	—
R9	WITHDRAWN 13 Sep 2026 — already done when this was written	A stale roadmap item. It proposed combining the better features with the four-component decomposition. The combination study (35–37, frozen at 7234964) had already tested exactly that, as arms C1 (variability), C2 (covariance) and C3 (both), against the same +0.05-on-both-participants criterion. All three are null: T11 $-0.034 / +0.030 / -0.047$, T5 $+0.020 / -0.124 / +0.017$. COMBINATION_STUDY_RESULT.md already said so — “that closes the last direction the results themselves suggested.” The audit listed as open a direction the project had already closed.	—	Nothing
R10	Unit tests for the core numerical functions	Added 13 Sep 2026, closing the gap this audit named in §6.5. 32 tests in tests/, each derived from an error the project actually made, plus tools/mutation_check.py, which reintroduces 11 historical bugs one at a time and confirms the matching test fails. Writing it surfaced a latent asymmetry: the exp ramp branch lacked the clip the linear branch had, so a leak there would have produced negative pre-onset values. No committed result is affected, because every episode uses the linear shape, and the branch is now clipped.	—	Nothing

14.3 The three most important next steps

Step	Why it is in the top three
1 Unblock the third participant (R1)	It is the only action that can change the project's binding limitation, and it costs almost nothing once the credentials exist. Every other scientific improvement is worth less until $n > 2$.
2 Decide the deadline and whether hardware gets built (R2)	Section 13 is a finished design sitting behind three decisions only the researcher can make — and one of them, the deadline, has a hard 6-week floor that gets harder to meet every day it goes unanswered.
3 Write the AI-use disclosure (R4)	It is low effort, it is the single highest-risk gap for competition, and the project has an unusually strong story to tell — fourteen documented errors, five withdrawn claims, three corrected p-values, and four debugged checking tools.

SECTION 15

15. Project memory / master context

This section is written to be read cold, by someone (or some future session) with no prior knowledge of the project. It is deliberately self-contained and deliberately repeats facts stated earlier.

15.1 What the project is, in five sentences

People with paralysis control a computer cursor through an implanted electrode array and a **decoder** that converts neural activity into motion. Over months the recording drifts and the decoder silently becomes wrong, and the only remedy — recalibration — is scheduled by guesswork. This project set out to build an early-warning system for that failure, found that **no observational dataset can validate such a system** because it cannot say when deterioration began, and therefore constructed a corpus of 1,850 degradation episodes whose onsets were chosen and locked before any detector existed. It then built a five-gate validation battery, built a monitor, and measured that **no configuration passes**. The reason is a single number: a recording session contains roughly one independent measurement, taken 55 times.

15.2 Standing constraints set by the researcher

These govern all future work on this project and are not negotiable by an assistant.

- **No scientific decisions made on the researcher's behalf**, and no analysis generated that the researcher does not understand.
- **No assumption of prior knowledge** of Python, pandas, Git, GitHub, or data-science terminology.
- **Errors are never hidden or silently worked around**. They are explained, along with the fix.
- **Raw data stays separate from processed data** and the original files are never modified.
- **Nothing is optimised for statistical significance**. Multiple deterioration definitions must not be tried and the best-performing one selected.
- **The current idea is not assumed to be the best version of the project**.
- **Everything is recorded in GitHub**, and the research log captures decisions **and mistakes**.
- **HARD CONSTRAINT: no human participants**. This includes the researcher — data collected from oneself is still human-participant research.
- **ISEF paperwork is deferred** until the researcher says otherwise.

15.3 The facts a future session must not have to rediscover

Fact	Value	Where
Repository / branch	gnair40/bci-ews-research / claude/isef-research-pipeline-9zt4uq	—
Dataset	MINDFUL, Pun et al. 2024, Dryad 10.5961/dryad.n2z34tn5s v6, CC0	data/raw/download_manifest.json
Participants	T11 (384 features, 15 sessions, days 658–800) and T5 (192 features, 6 sessions, days 2121–2149). T11 is the longitudinal one	reports/PHASE1_2_REPORT.md
Corpus	1,073 T11 episodes + 777 T5; 4 modes × 3 rates × 3 severities; master seed 20260826	data/processed/injection_plan*.json
Design freeze	2026-08-26 01:31:10 UTC at commit 20db485, with input SHA-256s and 2 amendments	research/FROZEN_DESIGN.json
The headline	0 of 48 configurations pass all five gates	reports/BENCHMARK_SUMMARY.md

Fact	Value	Where
The explanation	Lag-1 $r = 0.995$ (T11), 0.980 (T5); effective $n = 0.1$ and 0.4	reports/AGGREGATION_LIMIT.md
The gap	Achieved session-level AUC 0.673 (T11), 0.742 (T5). Needed: ≈ 0.99 at the $0.1/h$ budget as specified, or ≈ 0.93 for 80% detection at a 10% false-flag rate. Always say which target is meant	reports/OPERATING_POINT_BOUND.md
Gate order	Silence first , then rate-invariance, comparator, elapsed-time, detrend	research/phase3_design_implications.md
Reporting rule	Lead time and false-alarm rate are always reported as a pair , never singly	scripts/20_evaluation_harness.py
Unit rule	Bootstrap over episodes, never over windows . Window-level and episode-level numbers must be labelled as such	reports/UNIT_OF_ANALYSIS.md
Blocked on	Dryad credentials DRYAD_CLIENT_ID / DRYAD_CLIENT_SECRET for T15, Card et al. 2024, 10.5061/dryad.dncjsxm85, 11.6 GB — to be set in the environment config, not pasted in chat	research/research_log.md, 28 Aug entry

15.4 Working practices this project follows

1. **Preregister before computing.** Every study since Phase 3 has a frozen note or JSON stating its criterion — and, where possible, a prediction — committed before the analysis runs.
2. **Build the grader before the thing being graded.** The evaluation harness was committed when the only detectors it could score were three the project did not invent.
3. **Correct inline, never in place.** A corrected report carries a (!) block explaining what was wrong. The original text is not edited away.
4. **Run the gates, then commit.** Running them afterwards leaves data/processed/*.json dirty; a stop hook caught this once.
5. **Never re-date the log to match the commits.** Reconcile the mismatch in a gate instead, with the reason recorded.
6. **A checking tool must be tested against a defect already known to exist** before its output is trusted. Four tools in this project were wrong before they were useful.
7. **Use `git commit -F` - with a heredoc** — apostrophes in commit messages break shell quoting.
8. **A finding is a prompt, not a verdict.** Linter hits are reviewed individually and deliberate sites are listed with reasons, so the gate stays clean rather than noisy.

15.5 Claims: what may and may not be quoted

Status	Count	Meaning	Examples
ESTABLISHED	17	Survived its challenges; backed by a recomputed check	The reference decoder works (C01); 0 of 48 pass (C03); lag-1 $r = 0.995$ (C04); session accuracy tracks decoder error (C09)
LIMITATION	10	A measured property of the corpus or method, not a result	Severity valid only in aggregate (L01); window overlap costs 0.007 – 0.012 AUC (L02); pooled p-values (L09)
WITHDRAWN	5	Retracted on evidence — do not quote	W01–W05, listed in full in Section 8.2
UNANSWERABLE	1	This data cannot decide it — not the same as no	Whether the session effect is failed detection or degraded labels (U01)
EXPLORATORY	5	Not preregistered; hypothesis-generating only	Task directional concentration varies 50-fold (E01); the supervised separability bounds (E04, E06)

38 entries in total, covering 92 of 93 verifier checks. The one uncovered check (“hardest mode pair, T11”) is reported by the register itself as a coverage gap — it asks whether the evidence base and the story have drifted apart, and answers honestly.

15.6 If the project has to be defended in one paragraph

THE PARAGRAPH

This project asked whether decoder failure can be seen coming. It discovered that the question cannot be answered honestly with observational data, because lead time depends on a definition the analyst chooses and false-alarm rate cannot be estimated at all. So it built the missing ground truth — 1,850 degradations whose onsets were locked under checksum before any detector existed — and a validation standard whose first gate would have caught its own earlier mistake. Then it measured that no configuration passes, and found the reason: a recording session contains roughly one independent measurement, so no amount of averaging, no better decision rule, and no longer window closes the gap between the 0.67–0.74 achieved and the ~0.93 a usable operating point would need. **It produced a benchmark that did not previously exist, a test battery, a replicated demonstration that the monitor sees what counting spikes cannot, one clean deployment-relevant positive, and a quantified bound on what would have to change.** Along the way it withdrew five of its own claims, corrected three published p-values from significant to null, and debugged four of its own checking tools. That is a more useful contribution than a fragile positive would have been, and every step of it is reproducible.

APPENDIX A

Appendix A — every verified claim, recomputed

Regenerated during this audit by `python3 scripts/31_verify_claims.py`. Each row is recomputed from stored data files rather than read from a report; the “claimed” column is what the reports state and the “actual” column is what the data files give now. **All 93 matched.**

#	Check	Claimed	Actual	
1	Reference decoder, T11 held-out healthy error (deg)	54.600	54.554	ok
2	Reference decoder, T11 measured chance (deg)	90.700	90.692	ok
3	Reference decoder, T5 held-out healthy error (deg)	52.800	52.800	ok
4	decoder-guard operating point, T11 recent-normal	50.460	50.458	ok
5	decoder-guard false alarms per hour, T11	3.410	3.407	ok
6	Early-warning AUC, T11 recent-normal	0.693	0.693	ok
7	Early-warning AUC, T5 recent-normal	0.707	0.707	ok
8	Early-warning AUC, T11 calibrate-once (should be chance)	0.491	0.491	ok
9	Lag-1 autocorrelation of risk, T11 healthy	0.995	0.995	ok
10	lag-1 r with NO window overlap, T11 (C04 is not geometry)	0.902	0.902	ok
11	lag-1 r with NO window overlap, T5	0.784	0.784	ok
12	effective samples never exceeds 1, worst spacing (T5, no overlap)	0.850	0.846	ok
13	overlap between adjacent windows as published (1500/250)	0.833	0.833	ok
14	Lag-1 autocorrelation of risk, T5 healthy	0.980	0.980	ok
15	Task change vs real fault separation, day 658	15.600	15.605	ok
16	Configurations passing all five gates	0.000	0.000	ok
17	F3 spectral, T11 (cleared the bar here)	0.750	0.750	ok
18	F3 spectral, T5 (failed badly here)	0.556	0.556	ok
19	F0 control, T11 (current features, generic scorer)	0.570	0.570	ok
20	calibration T11, 20 windows (scattered)	0.654	0.653	ok
21	calibration T11, entire healthy record	0.648	0.648	ok
22	calibration T5, 20 windows (scattered)	0.740	0.740	ok
23	calibration T5, entire healthy record	0.739	0.739	ok
24	calibration T5, 40 windows CONTIGUOUS	0.740	0.740	ok
25	calibration T11, 20 windows CONTIGUOUS	0.665	0.665	ok
26	n=10 profile covariance is singular (min eigenvalue)	0.000	0.000	ok
27	n=20 profile covariance is well posed (min eigenvalue)	0.057	0.057	ok
28	staleness permutation p, T11 (naive said 0.003)	0.128	0.128	ok
29	staleness permutation p, T5	0.080	0.080	ok
30	day-to-day AUC spread, T11 (min)	0.319	0.319	ok
31	day-to-day AUC spread, T11 (max)	0.974	0.974	ok
32	l-squared across days, T11	0.856	0.856	ok
33	share of day variance that is sampling, T11	0.255	0.255	ok

#	Check	Claimed	Actual	
34	fault-mix confound, T5 (rho) — why T5 cannot corroborate	0.886	0.886	ok
35	fault-mix confound, T11 (rho) — day effect stands	0.050	0.050	ok
36	inverse-variance weighting bias, T11 (rho)	-0.913	-0.913	ok
37	cost of pooling episodes across days, T11	0.003	0.003	ok
38	P5 decoder error vs monitor AUC, T11 (rho)	-0.720	-0.720	ok
39	P5 p-value, T11 (Bonferroni threshold 0.01)	0.005	0.005	ok
40	P5 partial rho controlling window length, T11	-0.773	-0.773	ok
41	P4 drift came out with the WRONG sign, T11	0.582	0.582	ok
42	P1 healthy dispersion, T11 (null)	0.203	0.203	ok
43	L4 vs decoder error, T11 (sign reverses)	0.681	0.681	ok
44	L4 vs decoder error, T5 (opposite sign)	-0.943	-0.943	ok
45	L4 vs monitor AUC, T11 (stage 2 null)	-0.137	-0.137	ok
46	guard vs distribution_shift per-day AUC, T11	0.835	0.835	ok
47	guard vs mean_activity per-day AUC, T11	-0.060	-0.060	ok
48	distribution_shift vs decoder error, T11	-0.670	-0.670	ok
49	ceiling: baseline vs achieved fault damage, T11	-0.747	-0.747	ok
50	ceiling: damage vs monitor AUC, T11 (predicted +, got ~0)	0.181	0.181	ok
51	P5 controlling for achieved damage, T11 (strengthens)	-0.794	-0.794	ok
52	trace noise vs monitor AUC, T11 (ties P5 exactly)	-0.720	-0.720	ok
53	P5 controlling for trace noise, T11 (collapses)	-0.333	-0.333	ok
54	spurious crossings vs monitor AUC, T11	-0.665	-0.665	ok
55	T11 days scoreable on unambiguous faults (of 13)	7.000	7.000	ok
56	bad days retain fewer unambiguous faults, T11 (rho)	-0.604	-0.604	ok
57	P5 on unambiguous episodes only, T11 (underpowered)	-0.536	-0.536	ok
58	margin over own chance vs monitor AUC, T11	0.264	0.264	ok
59	directional concentration vs monitor AUC, T11	-0.516	-0.516	ok
60	day 800 barely beats its own chance, P(better)	0.531	0.531	ok
61	severity ladder monotone fraction, T11	0.567	0.567	ok
62	severity ladder monotone fraction, T5	0.667	0.667	ok
63	baseline vs ladder validity, T11 (rho)	-0.677	-0.677	ok
64	baseline vs ladder validity, T5 (rho)	-0.926	-0.926	ok
65	contaminated windows per episode, T11 (1500/250 = 6)	5.990	5.990	ok
66	AUC understated by the overlap, T11	0.007	0.007	ok
67	AUC understated by the overlap, T5	0.012	0.012	ok
68	sign test across both arrays, p (one-sided)	0.011	0.011	ok
69	attribution accuracy overall, T11 (chance 0.25)	0.563	0.563	ok
70	attribution accuracy overall, T5 (chance 0.33)	0.525	0.525	ok
71	GAIN_DRIFT named correctly, T11	0.993	0.993	ok
72	GEOMETRY_ROTATION named correctly, T11	0.000	0.000	ok

#	Check	Claimed	Actual	
73	GEOMETRY_ROTATION named correctly, T5	0.000	0.000	ok
74	during rotation, dispersion z (vs profile 1.92)	16.510	16.510	ok
75	during rotation, profile IS lit (fraction)	0.800	0.800	ok
76	gain-drift vs rotation separability, T11	0.920	0.919	ok
77	gain-drift vs rotation separability, T5	0.998	0.998	ok
78	hardest mode pair, T11 (the real ceiling)	0.574	0.575	ok
79	headline without GEOMETRY_ROTATION, T11	0.636	0.636	ok
80	headline without GEOMETRY_ROTATION, T5	0.740	0.740	ok
81	episode-level headline, T11 (register C02 is window-level)	0.672	0.672	ok
82	window-level headline reproduces the register, T11	0.693	0.693	ok
83	sample-size inflation from pooling windows	26.600	26.600	ok
84	published statistics that were $p = 0$ exactly	4.000	4.000	ok
85	results significant as published, null when corrected	3.000	3.000	ok
86	largest AUC shift from the correction	0.054	0.054	ok
87	confusable trio, invariant features, T11	0.987	0.987	ok
88	confusable trio, per-channel features, T11	0.650	0.650	ok
89	control: 12 RANDOM raw channels, T11	0.702	0.702	ok
90	rotation pairs barely moved, T11 (as predicted)	0.968	0.968	ok
91	invariant one-class detector AUC, T11 (worse)	0.617	0.617	ok
92	invariant one-class detector AUC, T5 (worse)	0.680	0.680	ok
93	healthy episodes trending, invariant detector T11	0.985	0.985	ok
94	same trend unclipped, T11 (not an artefact)	0.989	0.989	ok
95	label-free invariant attribution, T11 (worse)	0.511	0.511	ok
96	label-free invariant attribution, T5 (worse)	0.397	0.397	ok
97	rotation under the invariant rule, T11 (below chance)	0.021	0.021	ok

APPENDIX B

Appendix B — sources the project cites

Transcribed from `research/ISEF_RESEARCH_PLAN.md` §D, which states these are the sources the Rationale actually draws on. **The same section also states that a 25-source annotated review is in the project repository. It is not** — see Section 12.3.

#	Citation	DOI
1	Barrese, J. C., Rao, N., Paroo, K., Triebwasser, C., Vargas-Irwin, C., Franquemont, L., & Donoghue, J. P. (2013). Failure mode analysis of silicon-based intracortical microelectrode arrays in non-human primates. <i>Journal of Neural Engineering</i> , 10(6), 066014.	10.1088/1741-2560/10/6/066014
2	Helmich, M. A., Olthof, M., Oldehinkel, A. J., Wichers, M., Bringmann, L. F., & Smit, A. C. (2024). Slow down and be critical before using early warning signals in psychopathology. <i>Nature Reviews Psychology</i> , 3, 767–780.	10.1038/s44159-024-00369-y
3	Hughes, C. L., Flesher, S. N., Weiss, J. M., Downey, J. E., Collinger, J. L., & Gaunt, R. A. (2021). Long-term intracortical microelectrode array performance in a human: A 5 year retrospective analysis. <i>Journal of Neural Engineering</i> , 18(4).	10.1088/1741-2552/ac1add
4	Karpowicz, B. M., et al. (2025). Stabilizing brain-computer interfaces through alignment of latent dynamics. <i>Nature Communications</i> , 16.	10.1038/s41467-025-59652-y
5	Maturana, M. I., Meisel, C., Dell, K., Karoly, P. J., D'Souza, W., Grayden, D. B., Burkitt, A. N., Jiruska, P., Kudlacek, J., Hlinka, J., Cook, M. J., Kuhlmann, L., & Freestone, D. R. (2020). Critical slowing down as a biomarker for seizure susceptibility. <i>Nature Communications</i> , 11, 2172.	10.1038/s41467-020-15908-3
6	Pun, T. K., Khoshnevis, M., Hosman, T., et al. (2024). Measuring instability in chronic human intracortical neural recordings towards stable, long-term brain-computer interfaces. <i>Communications Biology</i>, 7. — the source of this project's data.	10.1038/s42003-024-06784-4 Dataset: 10.5061/dryad.n2z34tn5s
7	Scheffer, M., Bascompte, J., Brock, W. A., Brovkin, V., Carpenter, S. R., Dakos, V., Held, H., van Nes, E. H., Rietkerk, M., & Sugihara, G. (2009). Early-warning signals for critical transitions. <i>Nature</i> , 461, 53–59.	10.1038/nature08227
8	Sponheim, C., Papadourakis, V., Collinger, J. L., Downey, J., Weiss, J., Pentousi, L., Elliott, K., & Hatsopoulos, N. G. (2021). Longevity and reliability of chronic unit recordings using the Utah, intracortical multi-electrode arrays. <i>Journal of Neural Engineering</i> , 18(6), 066044.	10.1088/1741-2552/ac3eaf
9	van der Bolt, B., van Nes, E. H., & Scheffer, M. (2021). No warning for slow transitions. <i>Journal of the Royal Society Interface</i> , 18(174), 20200935.	10.1098/rsif.2020.0935
10	Wilkat, T., Rings, T., & Lehnertz, K. (2019). No evidence for critical slowing down prior to human epileptic seizures. <i>Chaos</i> , 29(9), 091104.	10.1063/1.5122759

Two further sources are named elsewhere in the repository and are not in §D: **Card et al. (2024)**, the T15 dataset identified as the third participant (10.5061/dryad.dncjxm85), and the MINDFUL analysis code the project reproduced at $r = 0.985$. Both belong in the bibliography if the plan is finalised.